EMBEDDED SYSTEMS

Version 2022 - UTFPR

PROF. DOUGLAS RENAUX, PHD



https://www.renesas.com/br/en/img/misc/products-solutions/synergy-ssp-title.jpg

14- RTOS – REAL-TIME OPERATING SYSTEM

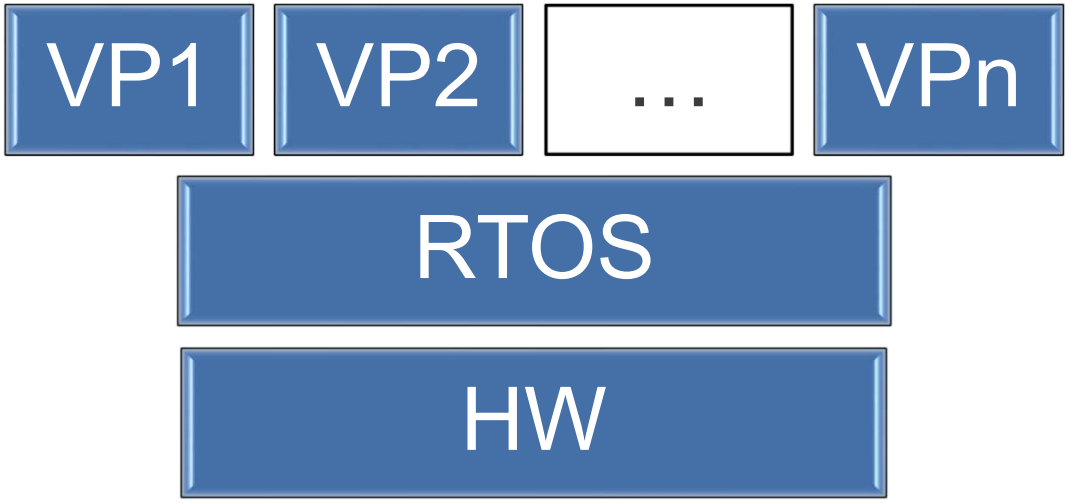
- Introduction
- Thread Management
- Inter-thread communication and synchronization
- Timing Services
- Memory Management

RTOS - INTRODUCTION

An RTOS is a layer of Software between the hardware and the application software that allows the implementation of concurrent tasks implemented by multithreading.

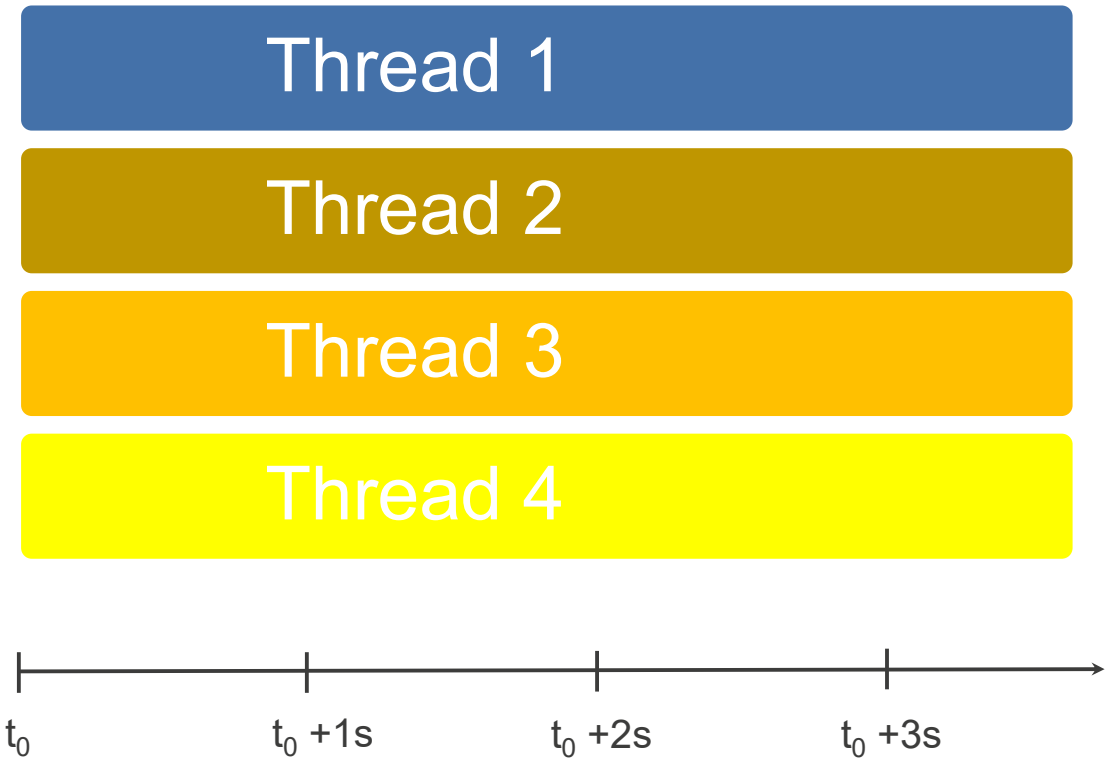
The RTOS manages the hardware, sharing resources among multiple “virtual processors”: VP1 to VPn so that each thread has the illusion of owning a processor and stack while sharing global data, heap, code area and peripherals.

The RTOS implements sharing of the processor via context switching and provides several important functionalities for the threads: timing, inter-thread communication and synchronization, and thread management.



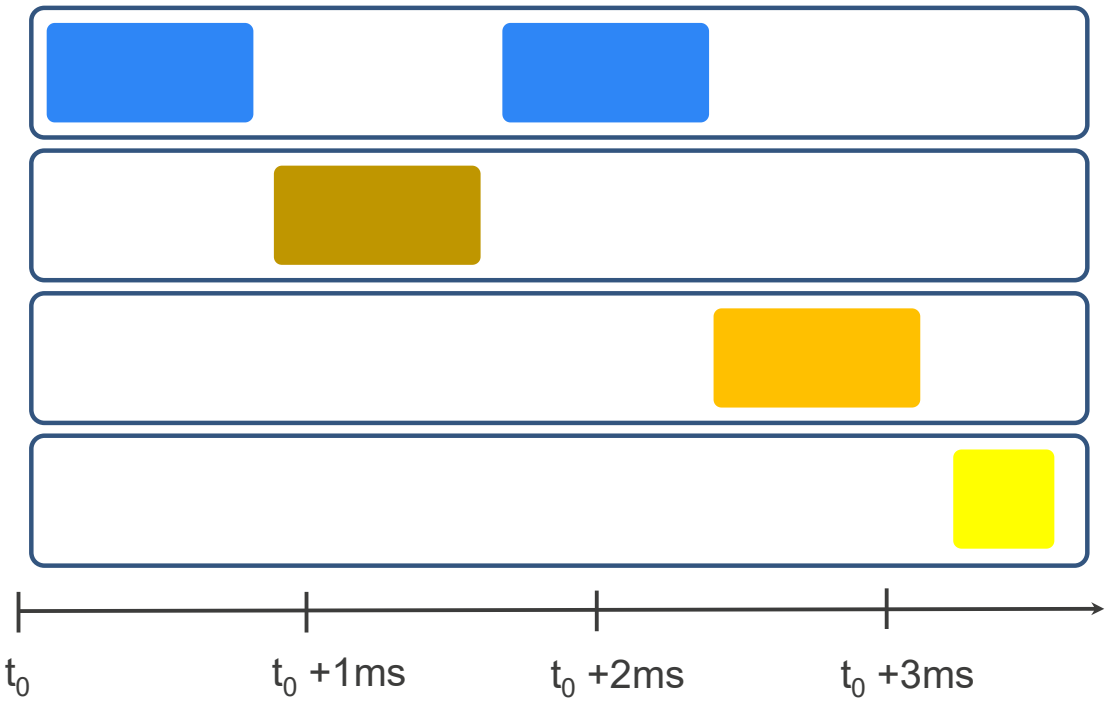
THREAD EXECUTION - LARGE GRAIN VIEW

Observing at a time scale of seconds, one has the impression that all threads execute in parallel.



THREAD EXECUTION - FINE GRAIN VIEW

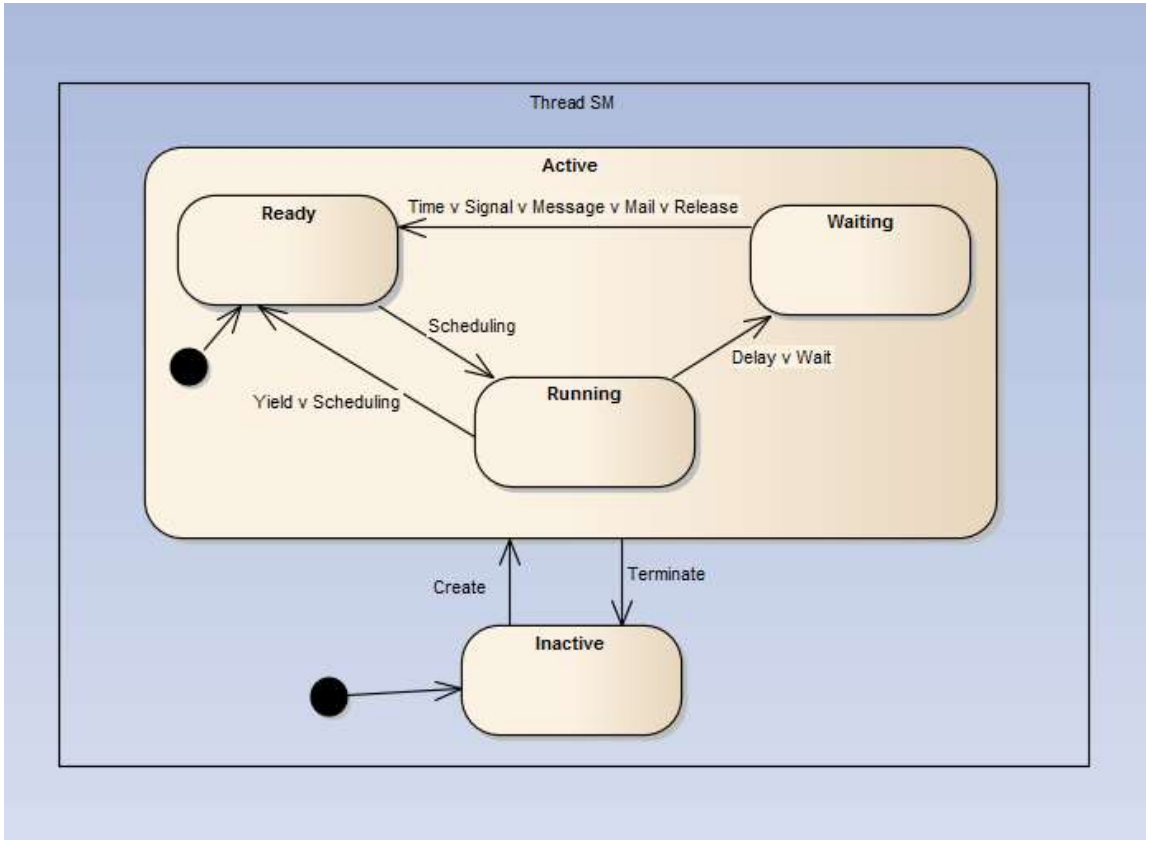
Observing on a fine grain scale (milliseconds) it is possible to realize that the tasks share the processor as the RTOS switches among them the utilization of the CPU.



TASK STATES

A typical state diagram of a task in a multithreaded environment:

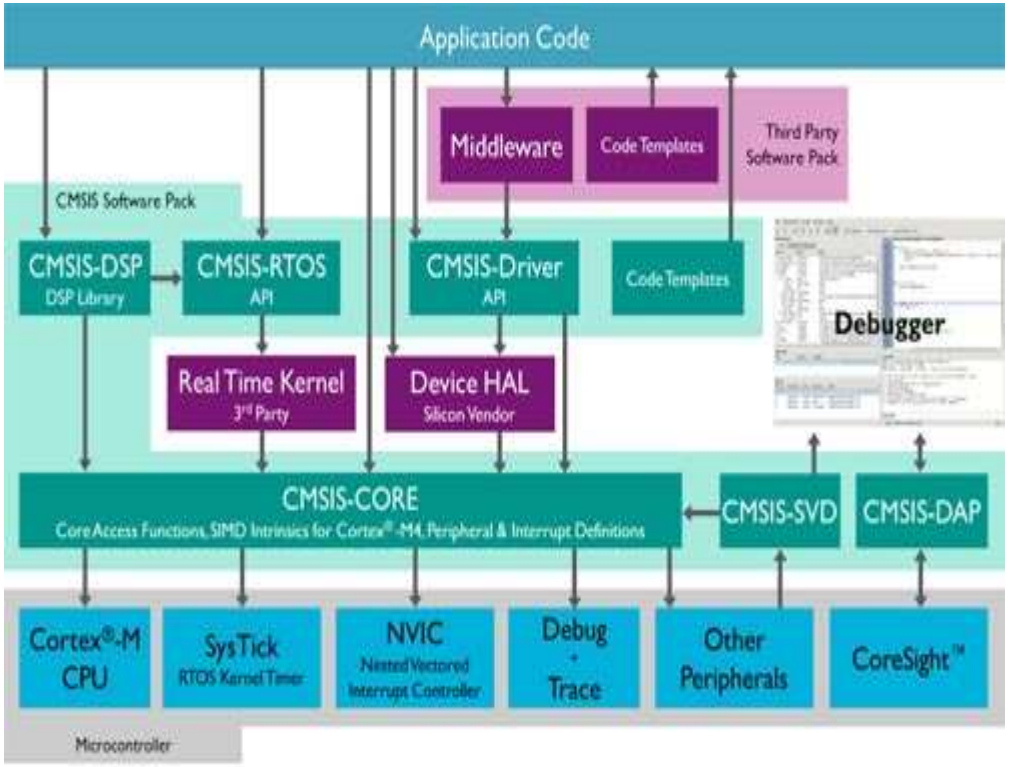
- **Ready**: tasks that is willing to use the processor and is expected to be scheduled.
- **Running**: the state of the task that is using the processor.
- **Waiting**: tasks whose execution is blocked due due to time, synchronization or communication.



CMSIS
(CORTEX MICROCONTROLLER SOFTWARE INTERFACE STANDARD)

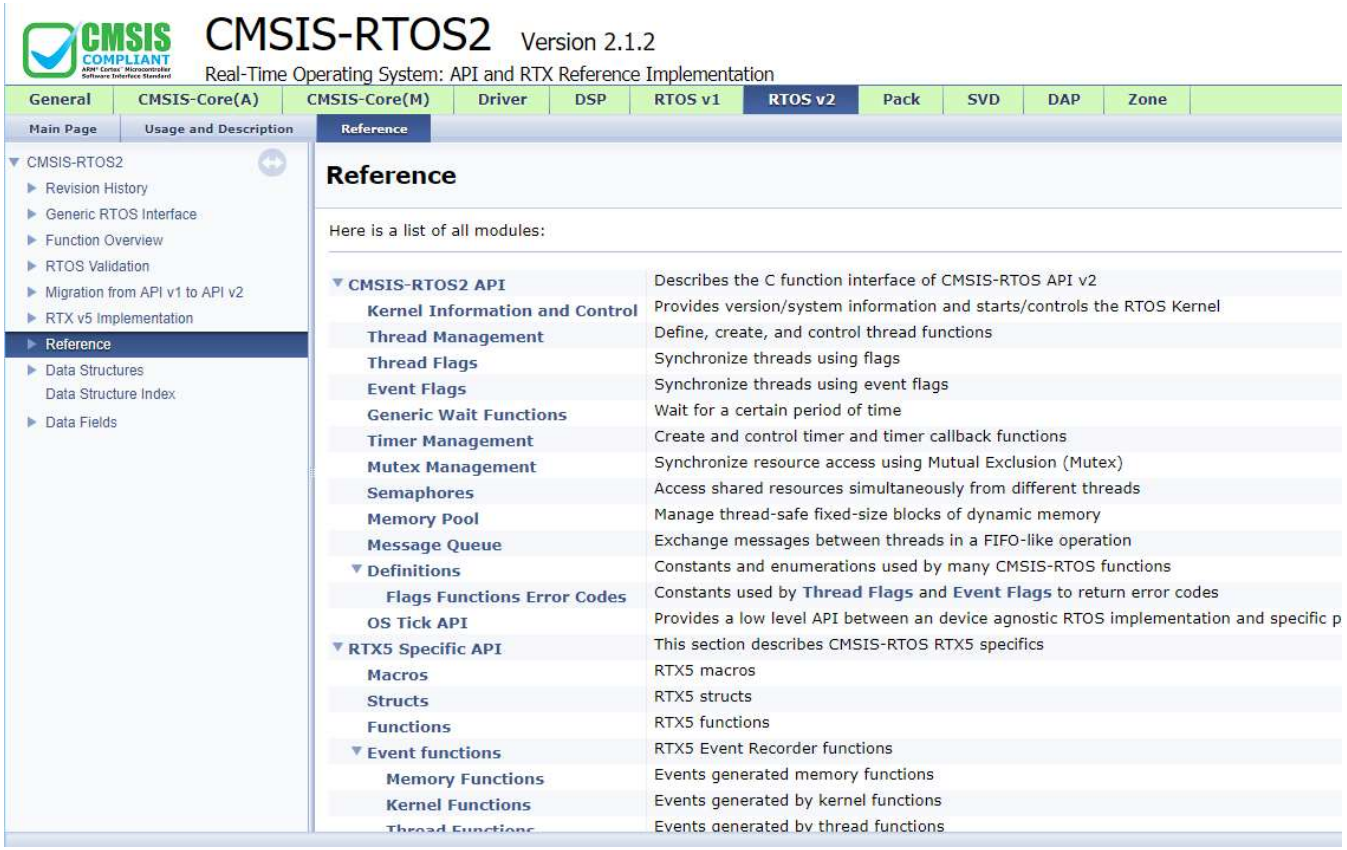
- v 1.0 (2008)
  - Drivers API
- v 2.0
  - DSP Lib
- v 3.0 (2012)
  - API RTOS
  - DAP (Debug)
- V 4.0 (2014)
  - CMSIS-Driver
  - CMSIS-Pack
- V 5.0 (2016)
  CMSIS-RTOS v2

As of March,2020, CMSIS is at version 5.6



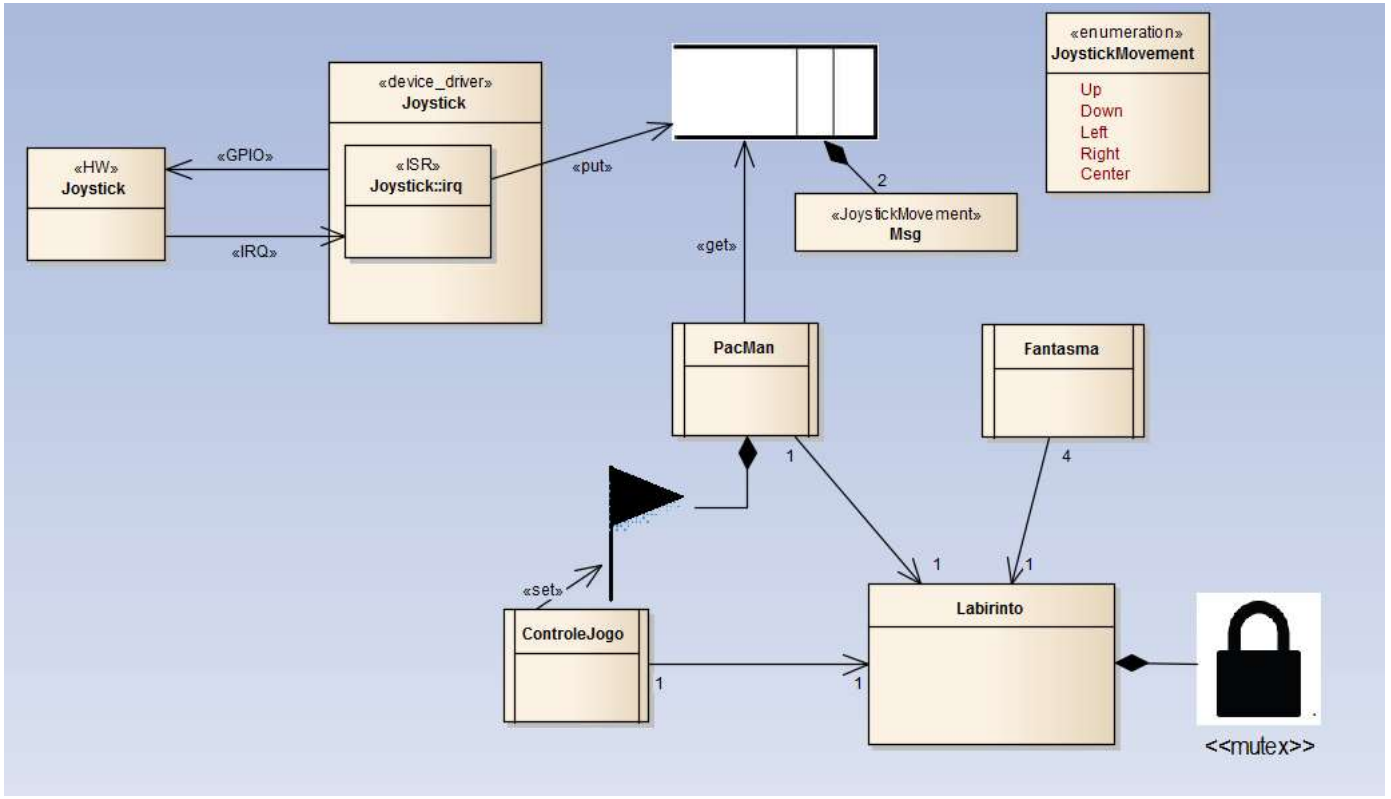
CMSIS-RTOS DOCUMENTATION

Available online



CMSIS code is available in GITHUB

PLANNING A MULTITHREADED APPLICATION



14- RTOS – THREADX

Express Logic 1997

Microsoft 2019

Available on github



Microsoft | Official Microsoft Blog Our Company News and Stories Press

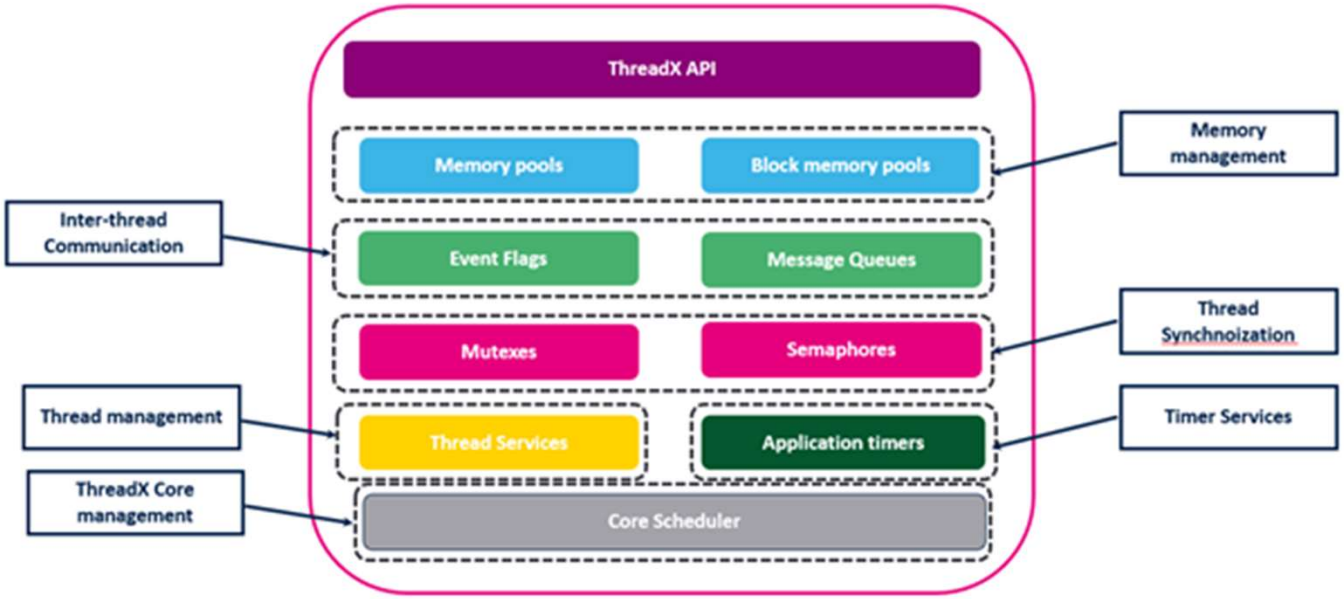
Microsoft acquires Express Logic, accelerating IoT development for billions of devices at scale

Apr 18, 2019 | Sam George - Corporate Vice President of Azure IoT

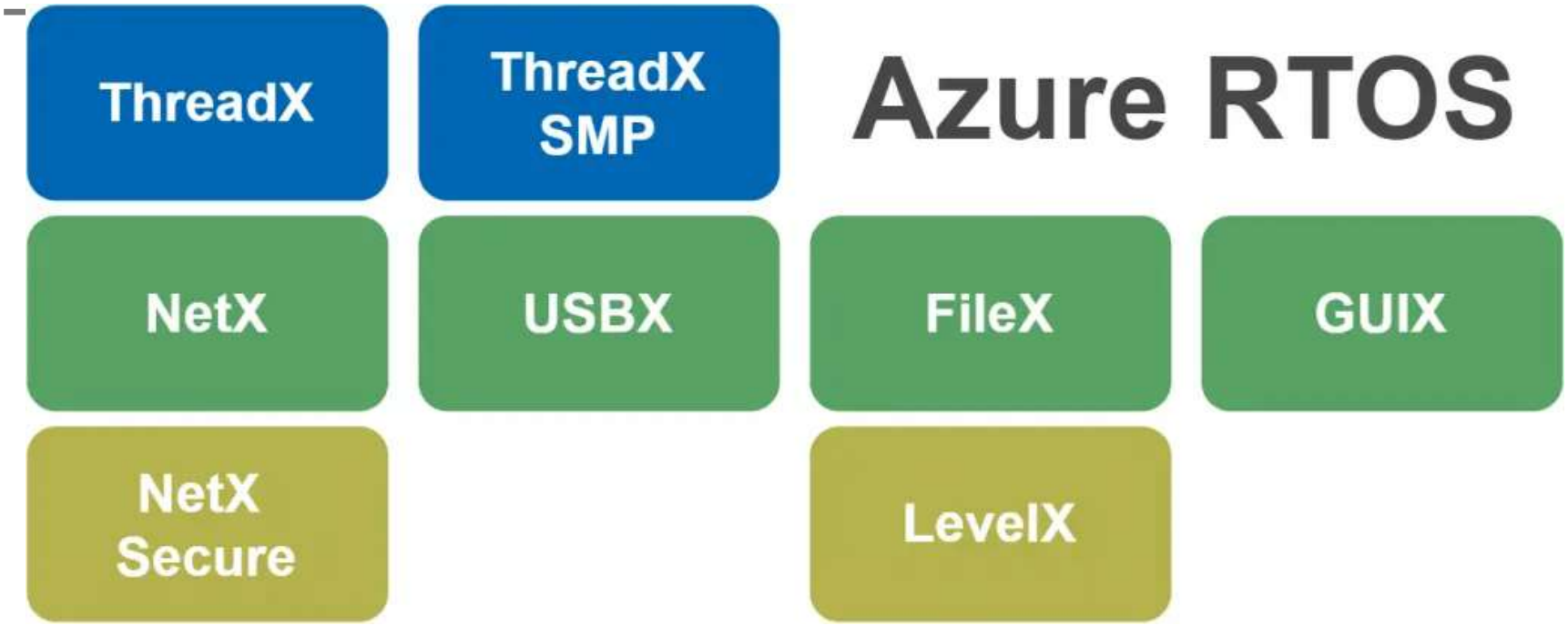
IoT sensors are being infused into just about everything, from industrial equipment to consumer devices, and increasingly these devices are connecting to the cloud. By 2020, Gartner predicts there will be more than 20 billion connected devices*. In April 2018, we announced we're investing $5 billion in IoT and the intelligent edge over the next four years. Since then, we've been making a number of investments from product innovation – including Azure Sphere, Azure Digital Twins, Azure IoT Edge, Azure Maps and Azure IoT Central – new partnerships with DJI, SAP, PTC, Qualcomm and Carnegie Mellon University for IoT and edge app development, and programs to help drive the next wave of innovation for our customers.

Today, I am incredibly excited to share we have acquired Express Logic, a leader in real time operating systems (RTOS) for IoT and edge devices powered by microcontroller units (MCUs). Express Logic's ThreadX RTOS has over 6.2 billion deployments, making it one of the most deployed RTOS in the world per VDC Research. This widespread popularity is driven by demand for technology to support resource constrained environments, especially those that require safety and security. Manufacturers building products across a range of categories – from low capacity sensors like lightbulbs

THREADX - KERNEL FUNCTIONALITY



AZURE RTOS THREADX SERVICE
Core Services (Require)
Queue Services
Event Flag Services
Semaphore Services
Mutex Services
Block Memory Services
Byte Memory Services


ThreadX
ThreadX SMP
Azure RTOS
NetX
USBX
FileX
GUIX
NetX Secure
LevelX

AZURE RTOS THREADX SERVICE	SERVICE TIME IN MICROSECONDS*
Thread Suspend	0.6
Thread Resume	0.6
Queue Send	0.3
Queue Receive	0.3
Get Semaphore	0.2
Put Semaphore	0.2
Context Switch	0.4
Interrupt Response	0.0 – 0.6

** Performance figures based on typical processor running at 200MHz.*

DATA TYPE	DESCRIPTION
UINT	Basic unsigned integer. This type must support 8-bit unsigned data; however, it is mapped to the most convenient unsigned data type.
ULONG	Unsigned long type. This type must support 32-bit unsigned data.
VOID	Almost always equivalent to the compiler's void type.
CHAR	Most often a standard 8-bit character type.

ThreadX type	Size
INT / UINT	32 bits
LONG / ULONG	32 bits
VOID	
CHAR / UCHAR	8 bits
SHORT / USHORT	16 bits

tx_port.h

```
/* Define ThreadX basic types for this port.  */

#define VOID                                    void
typedef char                                    CHAR;
typedef unsigned char                           UCHAR;
typedef int                                     INT;
typedef unsigned int                            UINT;
typedef long                                    LONG;
typedef unsigned long                           ULONG;
typedef short                                   SHORT;
typedef unsigned short                          USHORT;
```

FILENAME	DESCRIPTION
tx_api.h	C header file containing all system equates, data structures, and service prototypes.
tx_port.h	C header file containing all development-tool and targetspecific data definitions and structures.
demo_threadx.c	C file containing a small demo application.
tx.a (or tx.lib)	Binary version of the ThreadX C library that is distributed with the *standard* package.

tx_user.h

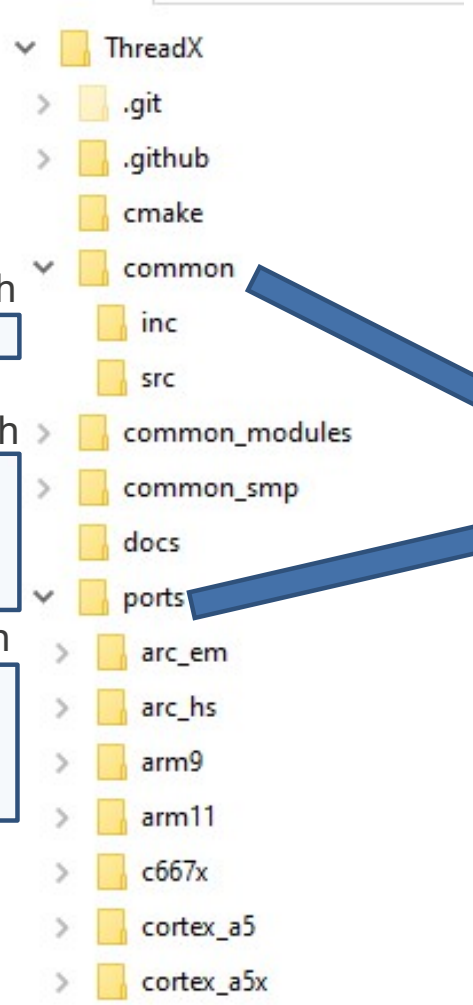
```
#define TX_TIMER_TICKS_PER_SECOND       (1000UL)
```

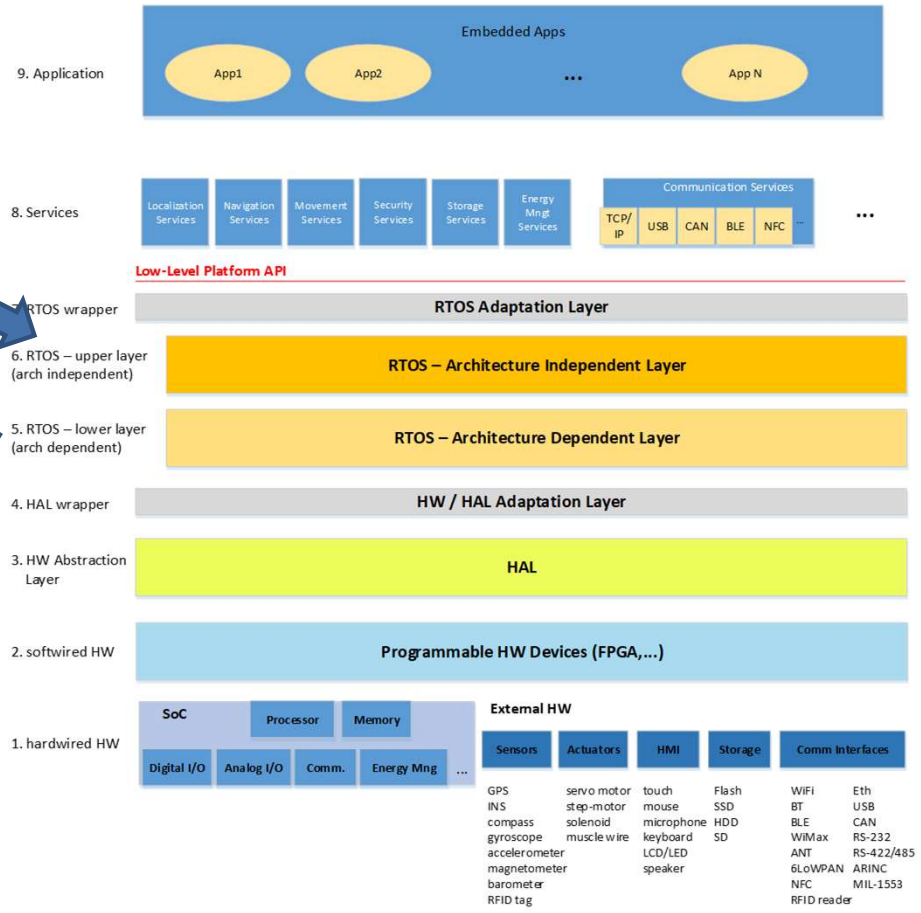
tx_port.h

```
#include "tx_user.h"
#ifndef TX_TIMER_TICKS_PER_SECOND
#define TX_TIMER_TICKS_PER_SECOND       (100UL)
#endif
```

tx_api.h

```
#include "tx_port.h"
#ifndef TX_TIMER_TICKS_PER_SECOND
#define TX_TIMER_TICKS_PER_SECOND       (100UL)
#endif
```





MINIMAL APPLICATION

There are four steps required to build a ThreadX application.

1. Include the ***tx_api.h*** file in all application files that use ThreadX services or data structures.
2. Create the standard C ***main*** function. This function must eventually call ***tx_kernel_enter*** to start ThreadX. Application-specific initialization that does not involve ThreadX may be added prior to entering the kernel.
3. Create the ***tx_application_define*** function. This is where the initial system resources are created. Examples of system resources include threads, queues, memory pools, event flags groups, mutexes, and semaphores.
4. Compile application source and link with the ThreadX run-time library ***tx.lib***. The resulting image can be downloaded to the target and executed!

```
#include "tx_api.h"
unsigned long my_thread_counter = 0;
TX_THREAD my_thread;
main( )
{
    /* Enter the ThreadX kernel. */
    tx_kernel_enter( );
}
void tx_application_define(void *first_unused_memory)
{
    /* Create my_thread! */
    tx_thread_create(&my_thread, "My Thread",
    my_thread_entry, 0x1234, first_unused_memory, 1024,
    3, 3, TX_NO_TIME_SLICE, TX_AUTO_START);
}
void my_thread_entry(ULONG thread_input)
{
    /* Enter into a forever loop. */
    while(1)
    {
        /* Increment thread counter. */
        my_thread_counter++;
        /* Sleep for 1 tick. */
        tx_thread_sleep(1);
    }
}
```

Made in SSP

TIPS

most of the ThreadX configurable files are generated by SSP, hence, cannot be edited directly. The solution is to include the source code of ThreadX in the project (Threads, HAL/Common, NewStack, X-Ware, ThreadX-source) then change properties.

CHANGING THREADX CONFIGURATION IN E2STUDIO

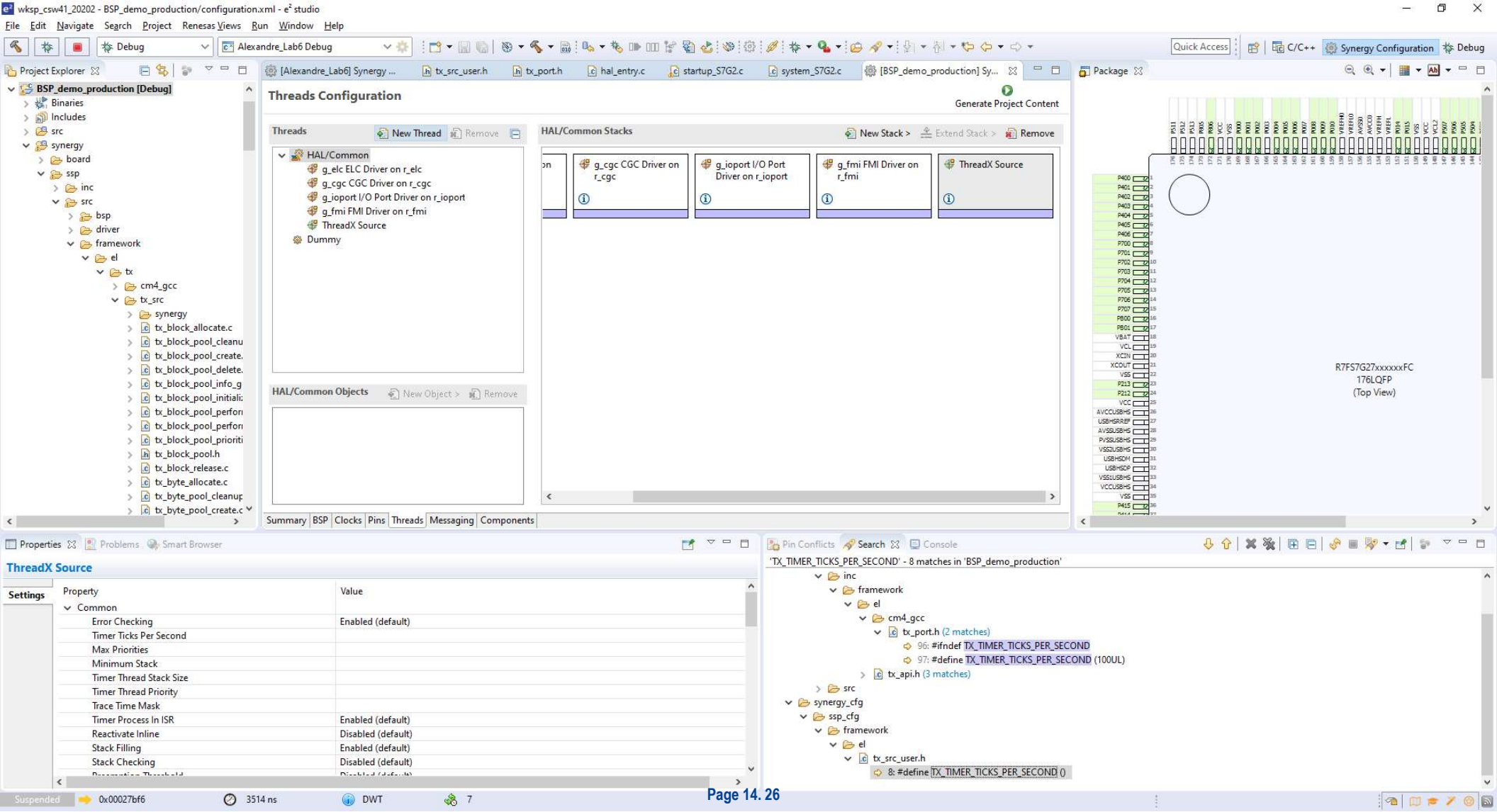
Most .h files are generated automatically by “Generate Project Content”

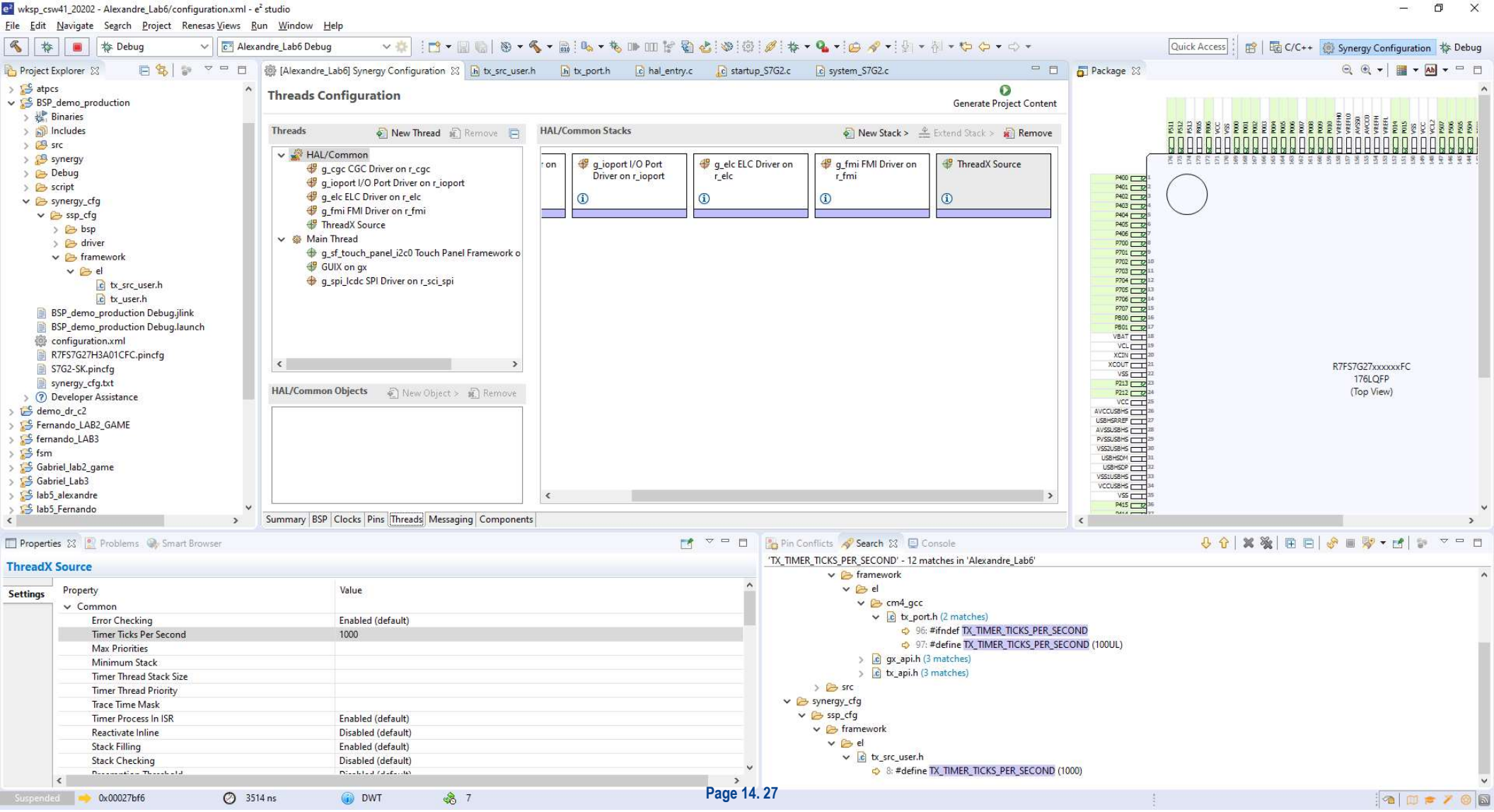
hence,

just editing a define in a .h has NO effect

Procedure:

- include ThreadX source code in HAL/Common
- open Properties tab
- edit desired property (e.g. Timer Ticks per Second)
- Generate Project Content
- in this example (timer ticks) the new value is defined in tx_src_user.h
- recompile (build)


wksp_csw41_20202 - BSP_demo_production/configuration.xml - e² studio
File Edit Navigate Search Project Renesas Views Run Window Help
Debug
Alexandre_Lab6 Debug
Quick Access
C/C++
Synergy Configuration
Debug
Project Explorer
BSP_demo_production [Debug]
Binaries
Includes
src
synergy
board
ssp
inc
src
bsp
driver
framework
el
tx
cm4_gcc
tx_src
synergy
tx_block_allocate.c
tx_block_pool_cleanu
tx_block_pool_create.
tx_block_pool_delete.
tx_block_pool_info_g
tx_block_pool_initiali:
tx_block_pool_perfor
tx_block_pool_perfor
tx_block_pool_prioriti
tx_block_pool.h
tx_block_release.c
tx_byte_allocate.c
tx_byte_pool_cleanup
tx_byte_pool_create.c
[Alexandre_Lab6] Synergy ...
tx_src_user.h
tx_port.h
hal_entry.c
startup_S7G2.c
system_S7G2.c
[BSP_demo_production] Sy...
Threads Configuration
Generate Project Content
Threads
New Thread
Remove
HAL/Common
g_elc ELC Driver on r_elc
g_cgc CGC Driver on r_cgc
g_ioport I/O Port Driver on r_ioport
g_fmi FMI Driver on r_fmi
ThreadX Source
Dummy
HAL/Common Objects
New Object >
Remove
HAL/Common Stacks
New Stack >
Extend Stack >
Remove
g_cgc CGC Driver on r_cgc
g_ioport I/O Port Driver on r_ioport
g_fmi FMI Driver on r_fmi
ThreadX Source
Summary
BSP
Clocks
Pins
Threads
Messaging
Components
Package
R7FS7G27xxxxxxFC
176LQFP
(Top View)
Properties
Problems
Smart Browser
ThreadX Source
Settings
Property
Value
Common
Error Checking
Enabled (default)
Timer Ticks Per Second
Max Priorities
Minimum Stack
Timer Thread Stack Size
Timer Thread Priority
Trace Time Mask
Timer Process In ISR
Enabled (default)
Reactivate Inline
Disabled (default)
Stack Filling
Enabled (default)
Stack Checking
Disabled (default)
Pin Conflicts
Search
Console
'TX_TIMER_TICKS_PER_SECOND' - 8 matches in 'BSP_demo_production'
inc
framework
el
cm4_gcc
tx_port.h (2 matches)
96: #ifndef TX_TIMER_TICKS_PER_SECOND
97: #define TX_TIMER_TICKS_PER_SECOND (100UL)
tx_api.h (3 matches)
src
synergy_cfg
ssp_cfg
framework
el
tx_src_user.h
8: #define TX_TIMER_TICKS_PER_SECOND ()
Suspended
0x00027bf6
3514 ns
DWT
7


wksp_csw41_20202 - Alexandre_Lab6/configuration.xml - e² studio
File Edit Navigate Search Project Renesas Views Run Window Help
Debug
Alexandre_Lab6 Debug
Quick Access
C/C++
Synergy Configuration
Debug
Project Explorer
atpcs
BSP_demo_production
Binaries
Includes
src
synergy
Debug
script
synergy_cfg
ssp_cfg
bsp
driver
framework
el
tx_src_user.h
tx_user.h
BSP_demo_production Debug.jlink
BSP_demo_production Debug.launch
configuration.xml
R7FS7G27H3A01CFC.pincfg
S7G2-SK.pincfg
synergy_cfg.txt
Developer Assistance
demo_dr_c2
Fernando_LAB2_GAME
fernando_LAB3
fsm
Gabriel_lab2_game
Gabriel_Lab3
lab5_alexandre
lab5_Fernando
[Alexandre_Lab6] Synergy Configuration
tx_src_user.h
tx_port.h
hal_entry.c
startup_S7G2.c
system_S7G2.c
Threads Configuration
Generate Project Content
Threads
New Thread
Remove
HAL/Common
g_cgc CGC Driver on r_cgc
g_ioport I/O Port Driver on r_ioport
g_elc ELC Driver on r_elc
g_fmi FMI Driver on r_fmi
ThreadX Source
Main Thread
g_sf_touch_panel_i2c0 Touch Panel Framework o
GUIX on gx
g_spi_lcdc SPI Driver on r_sci_spi
HAL/Common Objects
New Object >
Remove
HAL/Common Stacks
New Stack >
Extend Stack >
Remove
g_ioport I/O Port Driver on r_ioport
g_elc ELC Driver on r_elc
g_fmi FMI Driver on r_fmi
ThreadX Source
Summary BSP Clocks Pins Threads Messaging Components
Package
R7FS7G27xxxxxxFC
176LQFP
(Top View)
Properties
Problems
Smart Browser
ThreadX Source
Settings
Property
Value
Common
Error Checking
Enabled (default)
Timer Ticks Per Second
1000
Max Priorities
Minimum Stack
Timer Thread Stack Size
Timer Thread Priority
Trace Time Mask
Timer Process In ISR
Enabled (default)
Reactivate Inline
Disabled (default)
Stack Filling
Enabled (default)
Stack Checking
Disabled (default)
Pin Conflicts
Search
Console
'TX_TIMER_TICKS_PER_SECOND' - 12 matches in 'Alexandre_Lab6'
framework
el
cm4_gcc
tx_port.h (2 matches)
96: #ifndef TX_TIMER_TICKS_PER_SECOND
97: #define TX_TIMER_TICKS_PER_SECOND (100UL)
gx_api.h (3 matches)
tx_api.h (3 matches)
src
synergy_cfg
ssp_cfg
framework
el
tx_src_user.h
8: #define TX_TIMER_TICKS_PER_SECOND (1000)
Suspended
0x00027bf6
3514 ns
DWT
7

THREADX - THREAD STATES

```
UINT tx_thread_create(
    TX_THREAD *thread_ptr,
    CHAR *name_ptr,
    VOID (*entry_function)(ULONG),
    ULONG entry_input,
    VOID *stack_start,
    ULONG stack_size,
    UINT priority,
    UINT preempt_threshold,
    ULONG time_slice,
    UINT auto_start); //TX_AUTO_START or TX_DONT_START
```

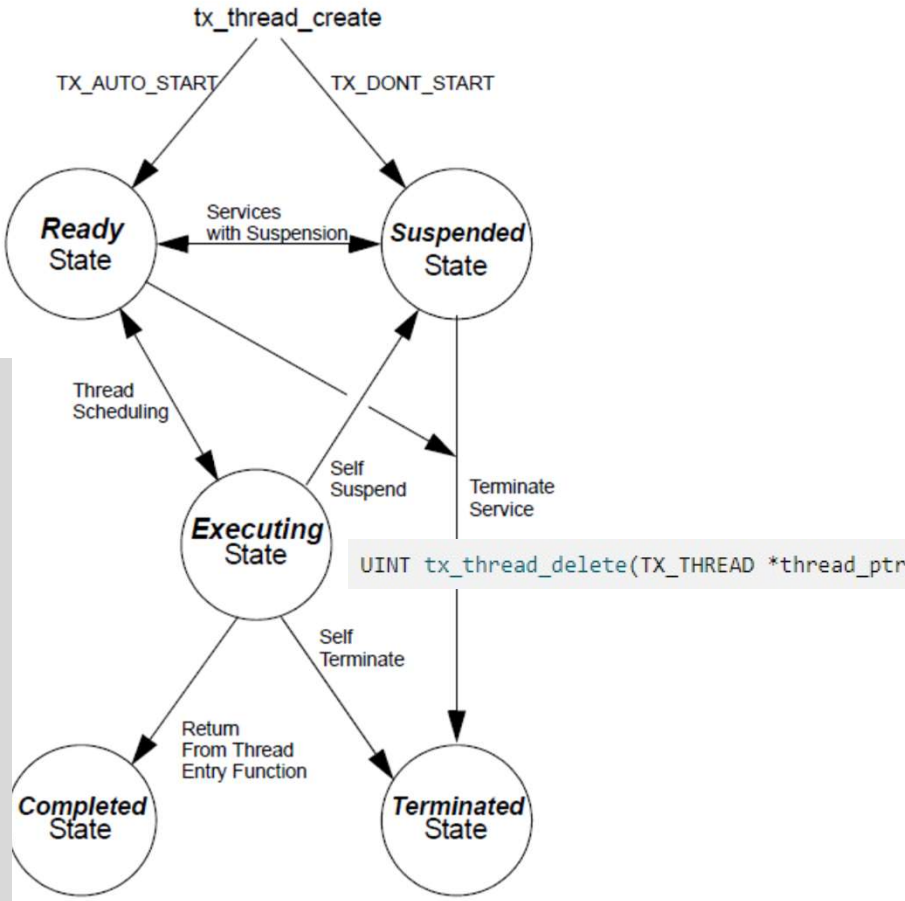


FIGURE 5. Thread State Transition

THREAD PRIORITIES

TX_MAX_PRIORITIES = number of priority levels

0 = highest priority

The ready thread with highest priority is executed first

Threads with same priorities are ordered FIFO

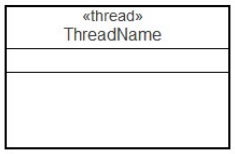
Cooperative Round-Robin (each thread calls tx_thread_relinquish) // Yield

Time-Slicing:

- establish a number of clock ticks (per thread at creation)
- when time slice expires, other threads of same priority execute

Priority Inheritance: within mutex services

FUNCTIONAL COMPONENTS OF THREADX

Functional Component	Symbol	Short description
Thread		multitasking primitive in concurrent programming
Memory Block (byte pool, block pool)	«memory block» 	memory block allocation primitive
Counting Semaphore	 « counting semaphore»	inter-task synchronization
Mutex	 «mutex»	controls mutual exclusion
Queue		inter-task communication
Event Flag		inter-task synchronization
Timer		thread temporization

THREADX - SEMAPHORE VS MUTEX

Semaphore	Mutex
Used for intertask synchronization, e.g. to indicate the occurrence of an event.	Used to implement Mutual Exclusion
Can count up to 32-bits (up to 4G)	Binary (but can count up to 4G)
There is **no ownership** of a semaphore.	At any given time, a **single thread may own** a mutex.
Operations: *tx_semaphore_get* and *tx_semaphore_put*	Operations: *tx_mutex_get* and *tx_mutex_put*
Any thread may perform a put	**Only the owner may perform a put**
Thread suspension when performing a get on a semaphore whose value is 0.	Thread suspension when performing a get on a mutex whose value is 0.
FIFO order of suspended threads	FIFO order of suspended threads
	May implement mutex priority inversion protocol

THREADX - BYTE POOL VS BLOCK POOL

Memory byte pool	Memory block pool
Thread-safe version of HEAP.	Thread-safe block allocator.
Operations are similar to malloc / free	Operations are similar to malloc / free
Multiple byte pools are possible.	Multiple block pools are possible. **Each block may be of different user-defined sizes.**
User defined allocation of the pool area in memory.	User defined allocation of the pool area in memory.
allocation/deallocation in byte increments	allocation/deallocation of a block
Fragmentation may occur.	**No fragmentation.**
Threads may suspend if they request memory, and it is not available.	Threads may suspend if they request memory, and it is not available.

THREADX SERVICES USAGE

```
UINT      tx_thread_create(TX_THREAD *thread_ptr,
            CHAR *name_ptr,
            VOID (*entry_function)(ULONG), ULONG entry_input,
            VOID *stack_start, ULONG stack_size,
            UINT priority, UINT preempt_threshold,
            ULONG time_slice, UINT auto_start);
```

Arguments to ThreadX services:

- Control structures

  Control structures use predefined ThreadX types. These control structures must be previously allocated by the programmer in data memory regions and passed (via pointers) to the ThreadX service.

- ASCII names

  Just ASCII strings with the name of the ThreadX element

- configuration options

  Typically, options are selected among elements of an enumeration/defines.

- outputs

  Typically, pointer to previously allocated memory areas where the results of this service call will be stored.

```
UINT      tx_thread_info_get(TX_THREAD *thread_ptr, CHAR **name,
            UINT *state, ULONG *run_count, UINT *priority,
            UINT *preemption_threshold, ULONG *time_slice,
            TX_THREAD **next_thread,
            TX_THREAD **next_suspended_thread);
```

tx_event_flags_create

Create event flags group

Prototype

C Copy

```
UINT tx_event_flags_create(
    TX_EVENT_FLAGS_GROUP *group_ptr,
    CHAR *name_ptr);
```

Description

This service creates a group of 32 event flags. All 32 event flags in the group are initialized to zero. Each event flag is represented by a single bit.

Parameters

- *group_ptr*: Pointer to an event flags group control block.
- *name_ptr*: Pointer to the name of the event flags group.

Return Values

- TX_SUCCESS (0x00) Successful event group creation.

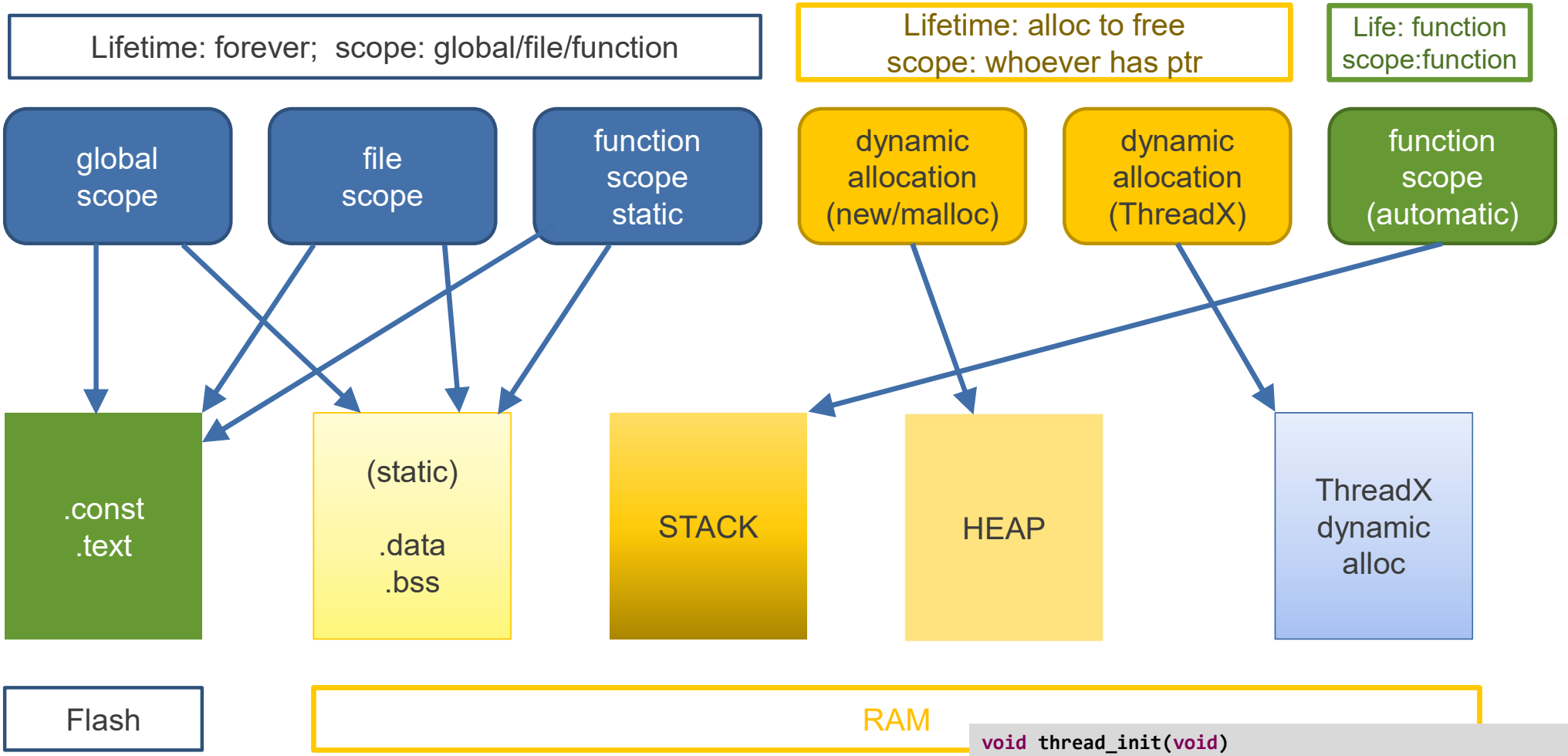
Allowed From

Initialization and threads

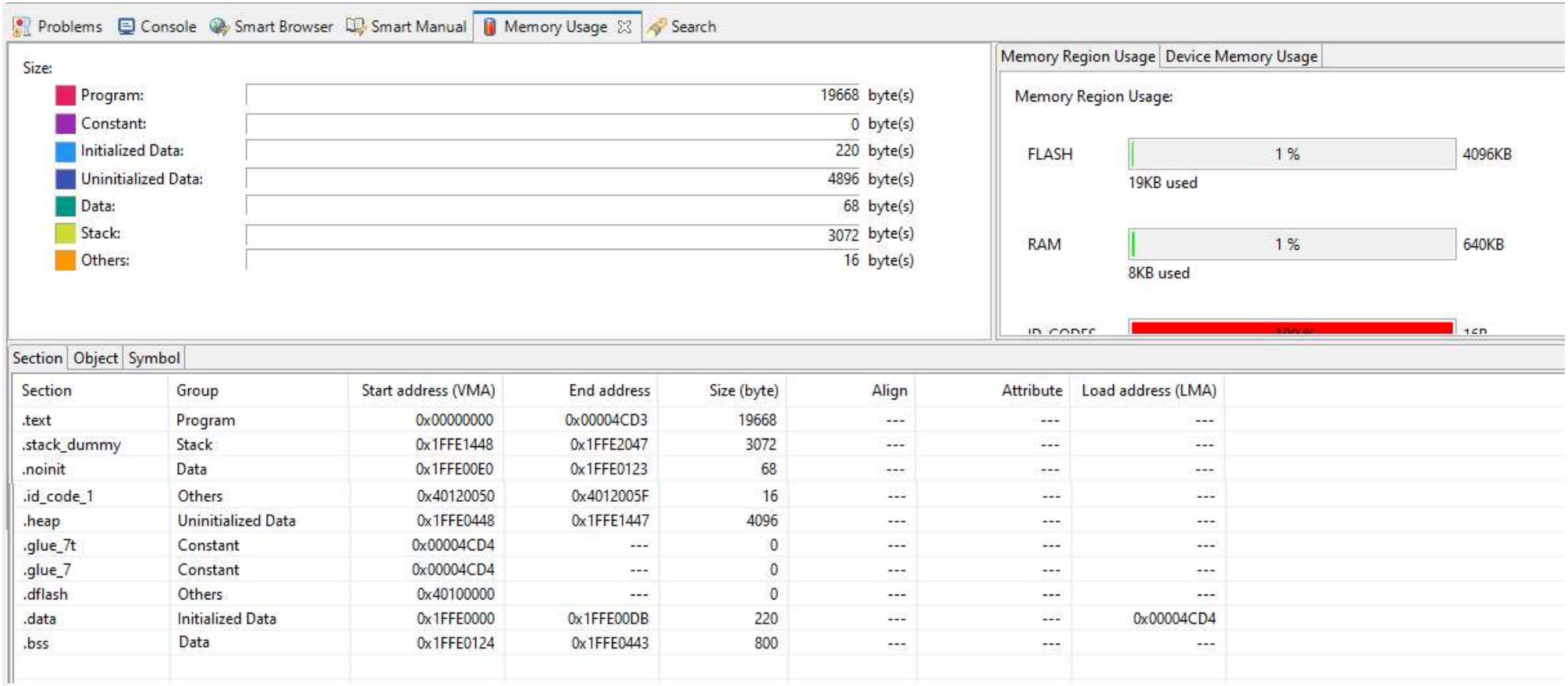
Preemption Possible

No






```
void thread_init(void)
{
    TX_EVENT_FLAGS_GROUP event_flags0;
    CHAR name[] = "Test1";
    tx_event_flags_create (&event_flags0, name);
}
```



Section	Group	Start address (VMA)	End address	Size (byte)	Align	Attribute	Load address (LMA)
.text	Program	0x00000000	0x00004CD3	19668	---	---	---
.stack_dummy	Stack	0x1FFE1448	0x1FFE2047	3072	---	---	---
.noinit	Data	0x1FFE00E0	0x1FFE0123	68	---	---	---
.id_code_1	Others	0x40120050	0x4012005F	16	---	---	---
.heap	Uninitialized Data	0x1FFE0448	0x1FFE1447	4096	---	---	---
.glue_7t	Constant	0x00004CD4	---	0	---	---	---
.glue_7	Constant	0x00004CD4	---	0	---	---	---
.dflash	Others	0x40100000	---	0	---	---	---
.data	Initialized Data	0x1FFE0000	0x1FFE00DB	220	---	---	0x00004CD4
.bss	Data	0x1FFE0124	0x1FFE0443	800	---	---	---

```
  Section            Kind         Address    Size  Object
  -------            ----         -------    ----  ------
"A0":                                         0x40
  .intvec            ro code          0x0    0x40  cstartup_M.o [1]
                                    - 0x40    0x40

"P1", part 1 of 2:                           0x138
  .text              ro code         0x40    0x54  tx_initialize_kernel_enter.o [7]
  .text              ro code         0x94    0xe0  txe_byte_pool_create.o [7]
  Initializer bytes  const          0x174     0x4  <for P2 s0>
                                    - 0x178   0x138

"P1", part 2 of 2:                          0x3818
  .text              ro code         0x410   0x470  sample_threadx.o [1]
  .text              ro code         0x880    0x98  txe_byte_allocate.o [7]
  .text              ro code         0x918   0x160  txe_thread_create.o [7]

  .rodata            const          0x3c28     0x0  zero_init3.o [4]
  .rodata            const          0x3c28     0x0  copy_init3.o [4]
                                    - 0x3c28  0x3818

"P2", part 1 of 3:                            0x4
  P2 s0                        0x2000'0000     0x4  <Init block>
    .data            inited    0x2000'0000     0x4  tx_thread_initialize.o [7]
                             - 0x2000'0004     0x4

"P2", part 2 of 3:                           0x3lcc
  .bss               zero      0x2000'0004  0x23a0  sample_threadx.o [1]
  .bss               zero      0x2000'23a4   0x400  tx_timer_initialize.o [7]
  .bss               zero      0x2000'27a4    0xd0  tx_iar.o [7]

"P2", part 3 of 3:                            0x400
  CSTACK                       0x2000'31d0   0x200  <Block>
    CSTACK           uninit    0x2000'31d0   0x200  <Block tail>
  HEAP                         0x2000'33d0   0x200  <Block>
    HEAP             uninit    0x2000'33d0   0x200  <Block tail>
                             - 0x2000'35d0   0x400

"P4":                                         0x10
  FREE_MEM           zero      0x2000'35d0    0x10  tx_initialize_low_level.o [1]
                             - 0x2000'35e0    0x10
```

.map file for IAR

STORAGE OF THREADX CONTROL STRUCTURES

Every functional component of ThreadX requires a corresponding control structure that must be created by the programmer and initialized by ThreadX using, typically, a *create* service.

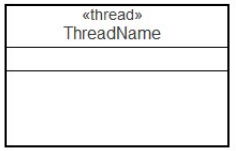
Where in memory should we to allocate such control structures ?

- Global RAM (.bss) for elements (threads, queues, event flags, ...) whose lifetime corresponds to the entire execution of the system (power-on to power-off).
- HEAP - likewise, provided malloc/new is executed before `tx_kernel_enter();`
- in memory regions allocated via `byte_pool`, if allocation occurs after `tx_kernel_enter()` or if we allocate and release during system's execution.

USAGE CYCLE OF THREADX FUNCTIONAL COMPONENTS

1. Allocate required data structures in an appropriate location:
   typically global, otherwise with adequate lifetime.
2. Initialize using create
3. Usage: main functions of each component
   e.g. for event_flags set and get
4. Usage: other functions
   performance, information, ...
5. Delete: rarely used.

FUNCTIONAL COMPONENTS OF THREADX

Functional Component	Symbol
Thread	
Memory Block (byte pool, block pool)	«memory block» 
Counting Semaphore	 « counting semaphore»
Mutex	 «mutex»
Queue	
Event Flag	
Timer	

USAGE CYCLE OF THREADX FUNCTIONAL COMPONENTS

Functional Component	Thread	Queue	Event Flag	Timer	Semaphore	Mutex	Block Pool	Byte Pool
Control Stucture	TX_THREAD	TX_QUEUE	TX_EVENT_FLAGS_GROUP	TX_TIMER	TX_SEMAPHORE	TX_MUTEX	TX_BLOCK_POOL	TX_BYTE_POOL
Create (initialize)	tx_thread_create	tx_queue_create	tx_event_flags_create	tx_timer_create	tx_semaphore_create	tx_mutex_create	tx_block_pool_create	tx_byte_pool_create
Usos	tx_thread_sleep	tx_queue_send (B/NB/TO)	tx_event_flags_get (B/NB/TO)	tx_timer_deactivate	tx_semaphore_get (B/NB/TO)	tx_mutex_get (B/NB/TO)	tx_block_allocate (B/NB/TO)	tx_byte_allocate (B/NB/TO)
	tx_thread_relinquish	tx_queue_receive (B/NB/TO)	tx_event_flags_set	tx_timer_change	tx_semaphore_put	tx_mutex_put	tx_block_release	tx_byte_release
	tx_thread_resume							

(B/NB/TO) - Blocking/Non-Blocking/Timeout

IMPORTANT INFORMATION

1. check the return values of the ThreadX calls.
2. check if a ThreadX call is being done from the appropriate/allowed context (e.g. a thread).
3. observe that several ThreadX calls may result in preemption.

Return Values

- **TX_SUCCESS** (0x00) Successful event flags group deletion.
- **TX_GROUP_ERROR** (0x06) Invalid event flags group pointer.
- **TX_CALLER_ERROR** (0x13) Invalid caller of this service.

Allowed From

Threads

Preemption Possible

Yes

IMPORTANT INFORMATION (CONT.)

- Many of the parameters to ThreadX call are pointers.
  Some of these pointers are **input** parameters, hence, they point to **previously** created data structures.
  Other pointers are **output** parameters and indicate **previously** allocated memory locations where results should be stored.
- The return value of ThreadX function calls is usually a status.
  Typically, a return of 0 (TX_SUCCESS) indicates that the call executed without error.

```
UINT err;
err = tx_thread_create(&hmi_thread, (CHAR*) "HMI Thread", hmi_thread_func,
        (ULONG) NULL, &hmi_thread_stack, 1024, 1, 1, 10, TX_AUTO_START);
if (TX_SUCCESS != err) {
        tx_startup_err_callback(&hmi_thread, 0);
}
```

Mutex Services

```
UINT      tx_mutex_create(TX_MUTEX *mutex_ptr, CHAR *name_ptr,
             UINT inherit);

UINT      tx_mutex_delete(TX_MUTEX *mutex_ptr);

UINT      tx_mutex_get(TX_MUTEX *mutex_ptr, ULONG wait_option);

UINT      tx_mutex_prioritize(TX_MUTEX *mutex_ptr);

UINT      tx_mutex_put(TX_MUTEX *mutex_ptr);
```

may suspend a thread

TX_NO_WAIT	(0x00000000)
TX_WAIT_FOREVER	(0xFFFFFFFF)
timeout value	(0x00000001 through 0xFFFFFFFE)

number of OS ticks

Return Values

TX_SUCCESS	(0x00)	Succ
TX_DELETED	(0x01)	Mute suspe
TX_NOT_AVAILABLE	(0x1D)	Service was unable to get ownership of the mutex within the specified time to wait.
TX_WAIT_ABORTED	(0x1A)	Suspension was aborted by another thread, timer, or ISR.
TX_MUTEX_ERROR	(0x1C)	Invalid mutex pointer.
TX_WAIT_ERROR	(0x04)	A wait option other than TX_NO_WAIT was specified on a call from a non-thread.
TX_CALLER_ERROR	(0x13)	Invalid caller of this service.

```
TX_MUTEX my_mutex;

UINT status;

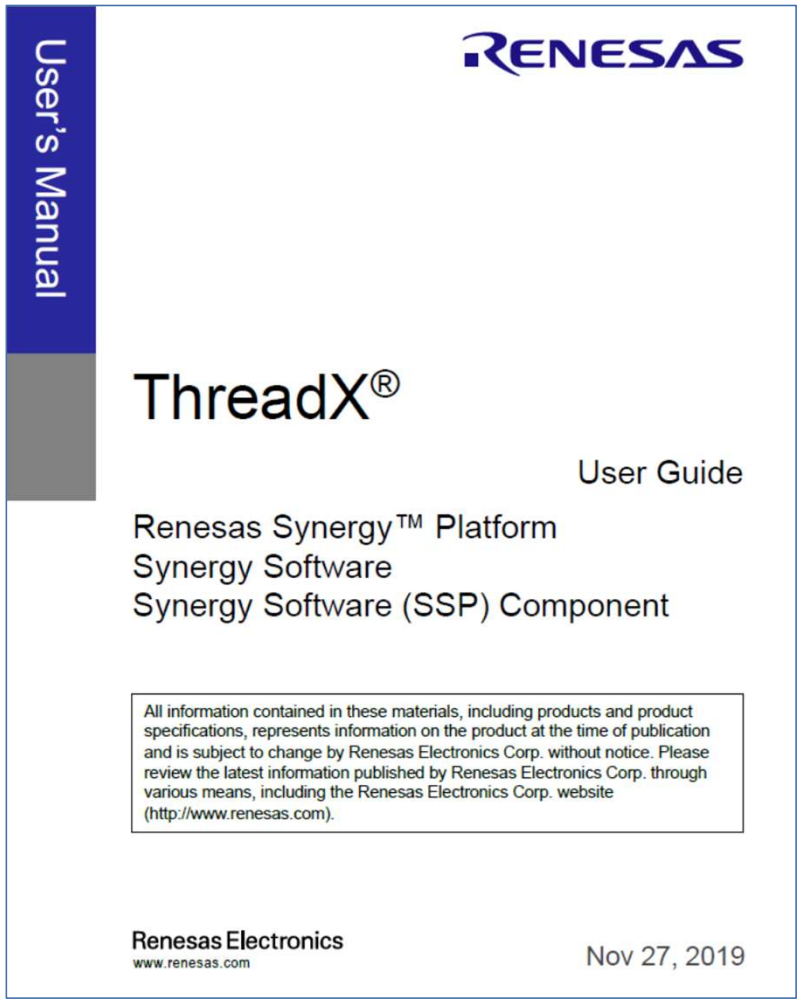
status = tx_mutex_create(&my_mutex,"my_mutex_name", TX_NO_INHERIT);

status = tx_mutex_get(&my_mutex, TX_WAIT_FOREVER);
```

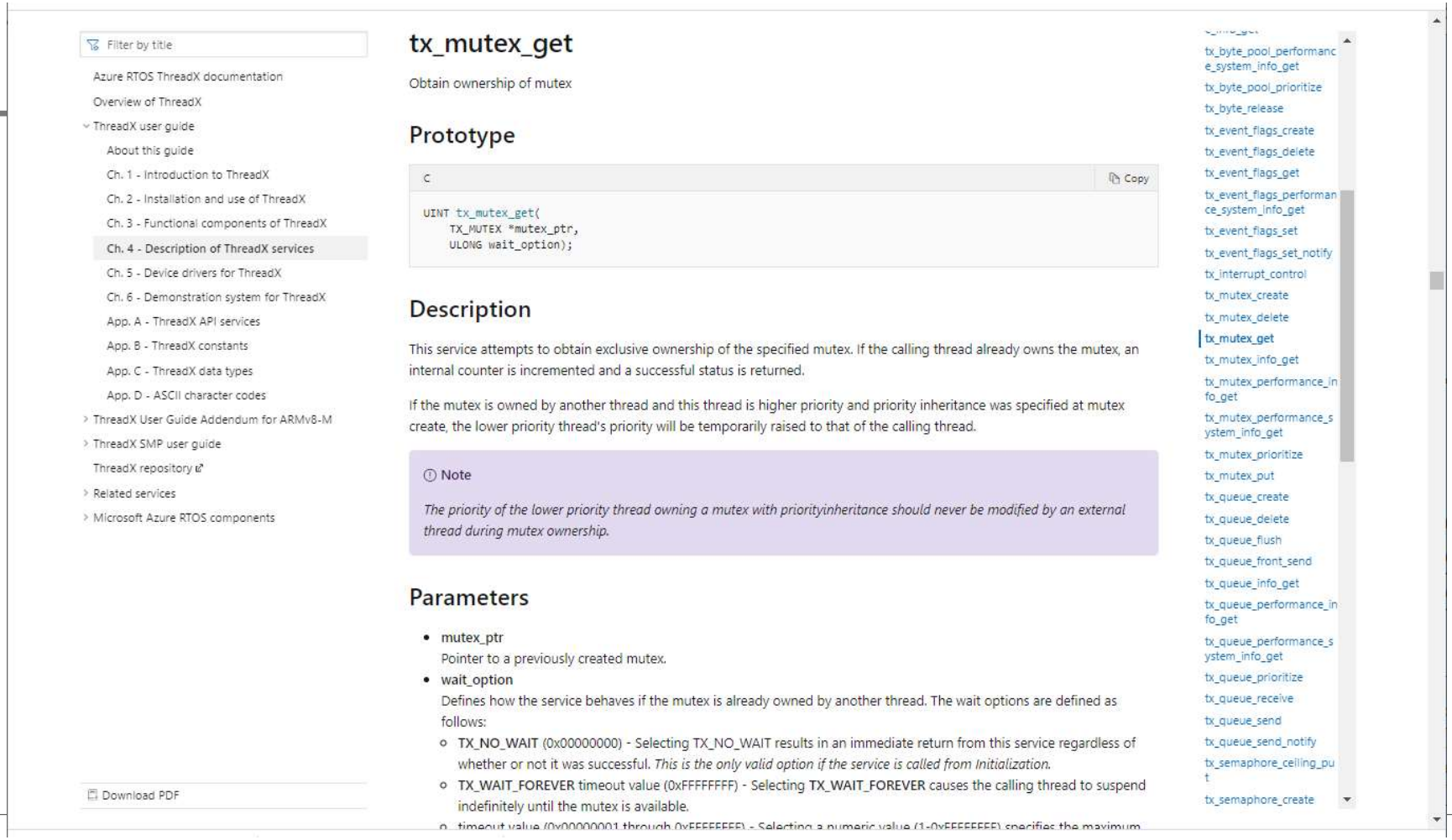
THREADX DOCUMENTATION

Renesas site: ThreadX pdf

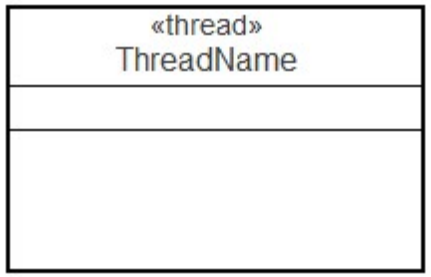
Microsoft site: Azure docs:
https://docs.microsoft.com/en-us/azure/rtos/threadx/chapter4#tx_mutex_get



https://docs.microsoft.com/en-us/azure/rtos/threadx/chapter4#tx_mutex_get



THREAD



- Control Structure: TX_THREAD
- Associated data structure: stack
- Associated functions: entry_function
- Scheduling related: priority, priority_threshold, time_slice

```
UINT      tx_thread_create(TX_THREAD *thread_ptr,
              CHAR *name_ptr,
              VOID (*entry_function)(ULONG), ULONG entry_input,
              VOID *stack_start, ULONG stack_size,
              UINT priority, UINT preempt_threshold,
              ULONG time_slice, UINT auto_start);
```

Thread Control Services

```
UINT      tx_thread_create(TX_THREAD *thread_ptr,
             CHAR *name_ptr,
             VOID (*entry_function)(ULONG), ULONG entry_input,
             VOID *stack_start, ULONG stack_size,
             UINT priority, UINT preempt_threshold,
             ULONG time_slice, UINT auto_start);

UINT      tx_thread_delete(TX_THREAD *thread_ptr);

UINT      tx_thread_entry_exit_notify(TX_THREAD *thread_ptr,
             VOID (*thread_entry_exit_notify)(TX_THREAD *, UINT));

TX_THREAD *tx_thread_identify(VOID);

UINT      tx_thread_info_get(TX_THREAD *thread_ptr, CHAR **name,
             UINT *state, ULONG *run_count, UINT *priority,
             UINT *preemption_threshold, ULONG *time_slice,
             TX_THREAD **next_thread,
             TX_THREAD **next_suspended_thread);

UINT      tx_thread_terminate(TX_THREAD *thread_ptr);

UINT      tx_thread_time_slice_change(TX_THREAD *thread_ptr,
             ULONG new_time_slice, ULONG *old_time_slice);

UINT      tx_thread_wait_abort(TX_THREAD *thread_ptr);
```

```
UINT      tx_thread_reset(TX_THREAD *thread_ptr);

UINT      tx_thread_resume(TX_THREAD *thread_ptr);

UINT      tx_thread_sleep(ULONG timer_ticks);

UINT      tx_thread_stack_error_notify
             VOID(*stack_error_handler)(TX_THREAD *));

UINT      tx_thread_suspend(TX_THREAD *thread_ptr);
```

TIMER



tx_event_flags_set

Set event flags in an event flags group

Allowed From

Initialization, threads, timers, and ISRs

- Control structure: TX_TIMER
- Associated Thread: System Timer
- Modes: single-shot or periodic
- Effect: call an expiration functions (runs in the System Timer's scope)
  Expiration function can send msg, **set event**, get/put semaphore, ...
- Counts down from the initial value (in ticks)
- Tick period is due to SysTick and can be changed (user_port.h or port.h)
- Single-shot deactivates when count ends; Periodic deactivates by calling tx_timer_deativate. Once inactive, can be reprogrammed (tx_timer_change).

Time Services

```
ULONG   tx_time_get(VOID);
VOID    tx_time_set(ULONG new_time);
```

Timer Services

```
UINT    tx_timer_activate(TX_TIMER *timer_ptr);

UINT    tx_timer_change(TX_TIMER *timer_ptr,
            ULONG initial_ticks,
            ULONG reschedule_ticks);

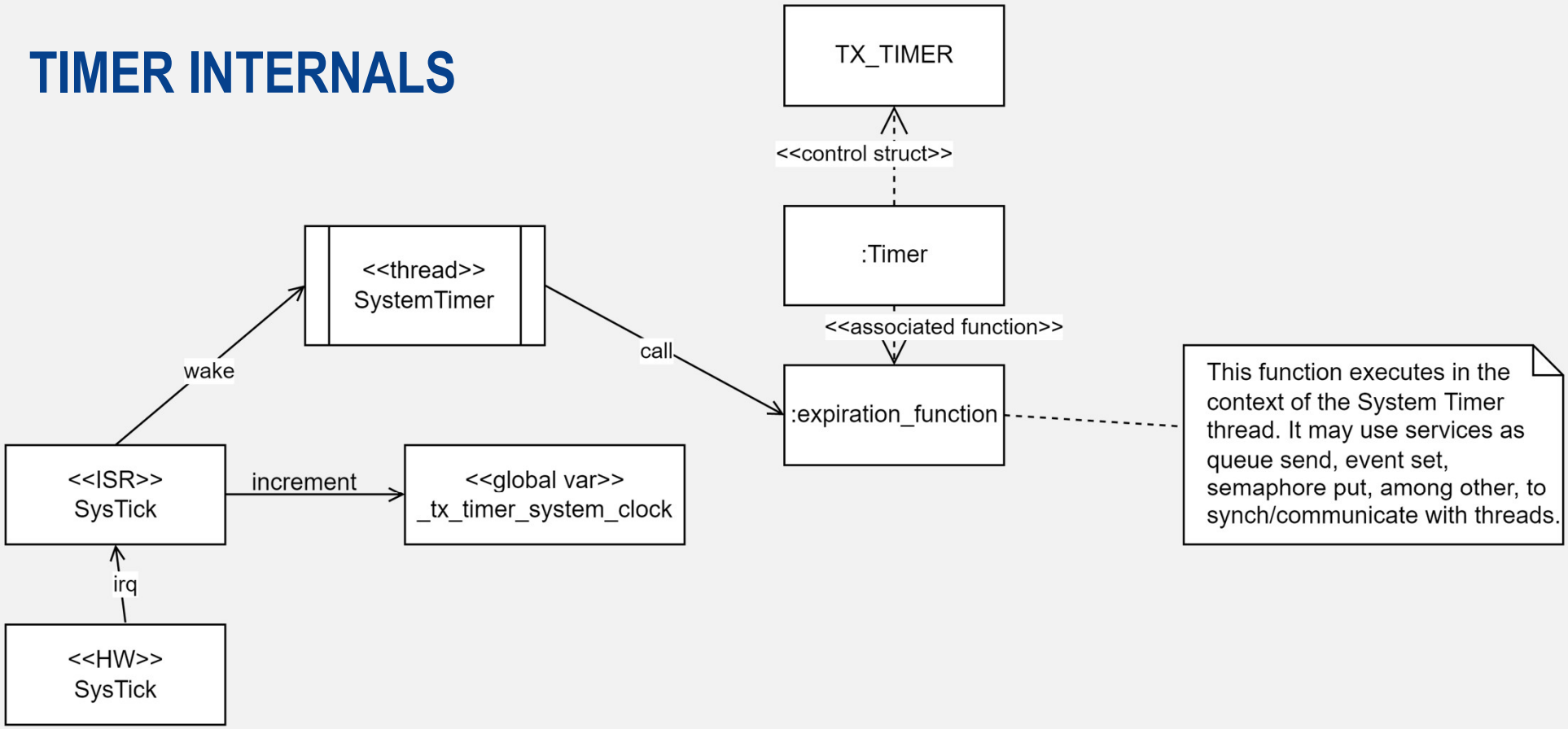
UINT    tx_timer_create(TX_TIMER *timer_ptr,
            CHAR *name_ptr,
            VOID (*expiration_function)(ULONG),
            ULONG expiration_input, ULONG initial_ticks,
            ULONG reschedule_ticks, UINT auto_activate);

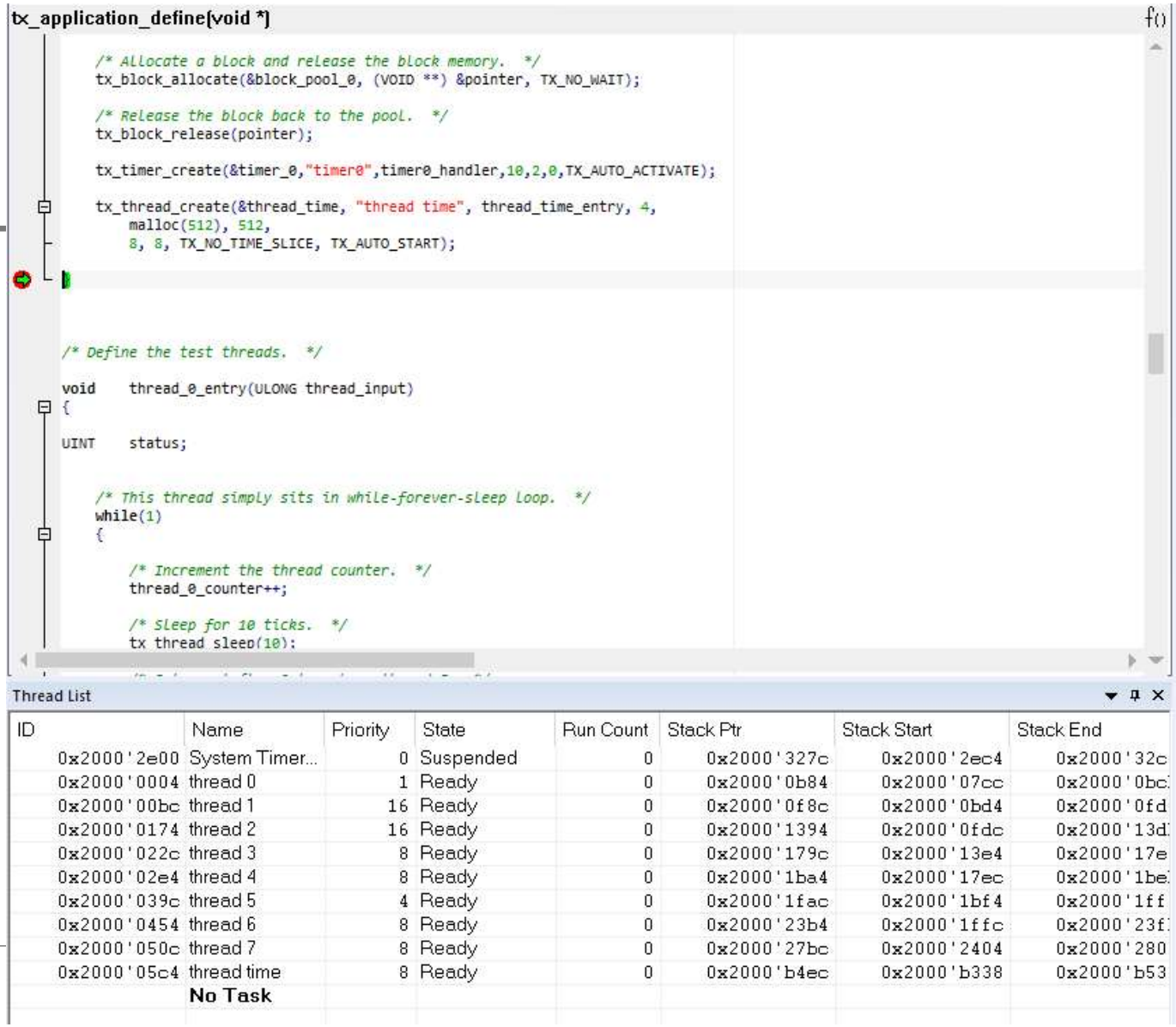
UINT    tx_timer_deactivate(TX_TIMER *timer_ptr);

UINT    tx_timer_delete(TX_TIMER *timer_ptr);

UINT    tx_timer_info_get(TX_TIMER *timer_ptr, CHAR **name,
            UINT *active, ULONG *remaining_ticks,
            ULONG *reschedule_ticks,
            TX_TIMER **next_timer);
```

TIMER INTERNALS




tx_application_define(void *)
/* Allocate a block and release the block memory. */
tx_block_allocate(&block_pool_0, (VOID **) &pointer, TX_NO_WAIT);
/* Release the block back to the pool. */
tx_block_release(pointer);
tx_timer_create(&timer_0,"timer0",timer0_handler,10,2,0,TX_AUTO_ACTIVATE);
tx_thread_create(&thread_time, "thread time", thread_time_entry, 4,
malloc(512), 512,
8, 8, TX_NO_TIME_SLICE, TX_AUTO_START);
/* Define the test threads. */
void thread_0_entry(ULONG thread_input)
{
UINT status;
/* This thread simply sits in while-forever-sleep loop. */
while(1)
{
/* Increment the thread counter. */
thread_0_counter++;
/* Sleep for 10 ticks. */
tx_thread_sleep(10);
Thread List
ID Name Priority State Run Count Stack Ptr Stack Start Stack End
0x2000'2e00 System Timer... 0 Suspended 0 0x2000'327c 0x2000'2ec4 0x2000'32c
0x2000'0004 thread 0 1 Ready 0 0x2000'0b84 0x2000'07cc 0x2000'0bc
0x2000'00bc thread 1 16 Ready 0 0x2000'0f8c 0x2000'0bd4 0x2000'0fd
0x2000'0174 thread 2 16 Ready 0 0x2000'1394 0x2000'0fdc 0x2000'13d
0x2000'022c thread 3 8 Ready 0 0x2000'179c 0x2000'13e4 0x2000'17e
0x2000'02e4 thread 4 8 Ready 0 0x2000'1ba4 0x2000'17ec 0x2000'1be
0x2000'039c thread 5 4 Ready 0 0x2000'1fac 0x2000'1bf4 0x2000'1ff
0x2000'0454 thread 6 8 Ready 0 0x2000'23b4 0x2000'1ffc 0x2000'23f
0x2000'050c thread 7 8 Ready 0 0x2000'27bc 0x2000'2404 0x2000'280
0x2000'05c4 thread time 8 Ready 0 0x2000'b4ec 0x2000'b338 0x2000'b53
No Task

SEMAPHORE



« counting semaphore»

- Control structure: TX_SEMAPHORE
- Purpose: Inter-thread synchronization
- Values 0 .. 4G
- Behavior: **get** decrements count; may suspend if 0 (B/NB/TO).
  put increments count; may ready a thread that was suspended.

Return Values

- **TX_SUCCESS** (0x00) Successful retr
- **TX_DELETED** (0x01) Counting sem
- **TX_NO_INSTANCE** (0x0D) Service
  counting semaphore (semaphore
- **TX_WAIT_ABORTED** (0x1A) Suspen
  ISR.
- **TX_SEMAPHORE_ERROR** (0x0C) In
- **TX_WAIT_ERROR** (0x04) A wait op
  call from a non-thread.

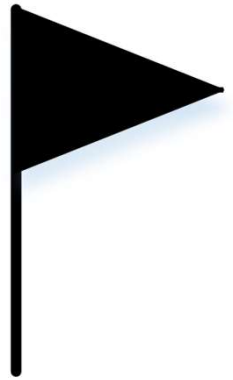
MUTEX



«mutex»

- Control structure: TX_MUTEX
- Purpose: Control Mutual Exclusion
- Ownership: a **single** thread can own a Mutex
  0 = resource is free
  >0 = resource is taken by a thread
- Values 0 .. 4G (several puts by the **same** thread)
- Only threads can own a mutex. Hence, not allowed for ISRs, timers, ...

EVENT FLAGS



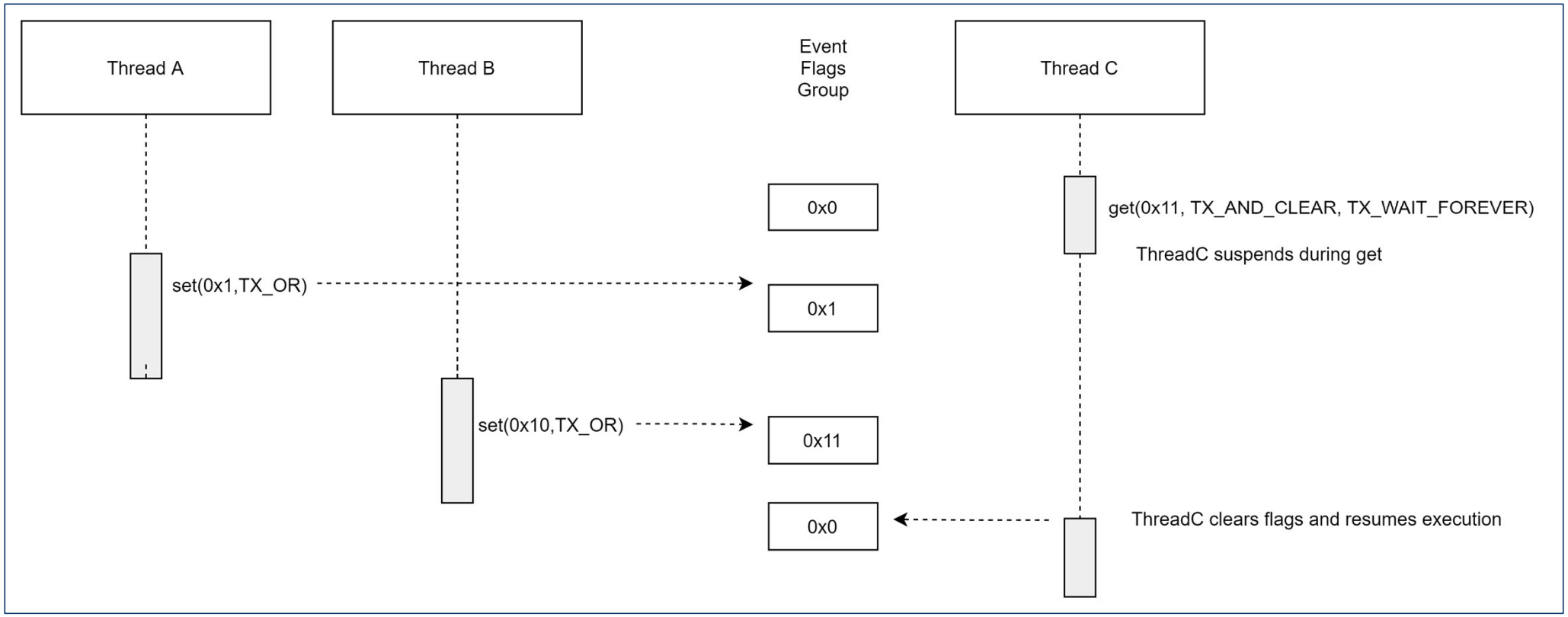
- Used for synchronization among threads.
- Each event flag is a single bit and is represented by a flag.
- The flag may be either up (set 1) or down (reset 0)
- Thread A informs Thread B of a given event by raising the corresponding flag.
- The receiving thread (Thread B) may wait in a suspended state and is made ready when the flag is set by Thread A.

EVENT FLAG GROUPS

- In ThreadX, flags are created in groups of 32.
- The control structure for a group of flags is of type `TX_EVENT_FLAGS_GROUP`.
- The `set` operation allows that a word be either `AND`ed or `OR`ed with the current state of the Flags Group.
- The `get` operation allows a thread to test for specific flags in the group, applying either `AND` or `OR`, if the test is successful, the corresponding flags may be cleared. This thread may request to be suspended until the desired flags are set. Such a request may have a timeout.

TX_AND	(0x02)
TX_OR	(0x00)

TX_AND	(0x02)
TX_AND_CLEAR	(0x03)
TX_OR	(0x00)
TX_OR_CLEAR	(0x01)



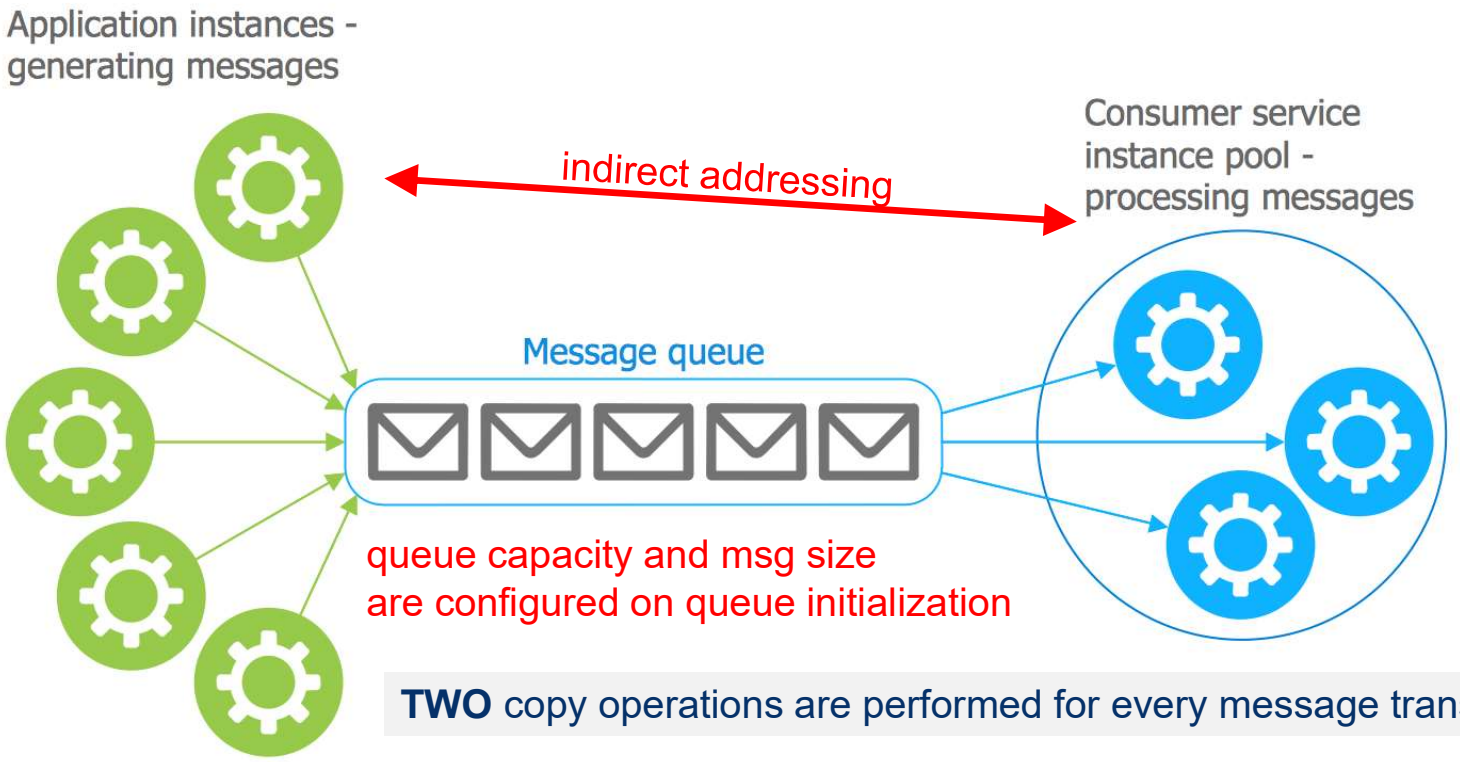
MESSAGE QUEUES IN THREADX



Operation:

- a queue control structure (TX_QUEUE) must be initialized with a given message size and message capacity; as well as a data storage area.
- message sizes can be configured from 1 to 16 words (word = 32-bits)
- **Send** - **copies** a message to the queue and proceeds, hence, non-blocking; however, may block if queue is full.
- **Receive** - **copies** the message at the head of the queue and proceeds; may block if queue is empty.
- Configurable blocking behavior:
  - NONBLOCKING - returns an error or success status;
  - BLOCKING - waits undefinedly for the blocking condition to be resolved;
  - BLOCKING with TIMEOUT - specifies a maximum wait time in TICKS.

MESSAGE QUEUE



tx_queue_create 180
Create message queue

tx_queue_delete 182
Delete message queue

tx_queue_flush 184
Empty messages in message queue

tx_queue_front_send 186
Send message to the front of queue

tx_queue_info_get 188
Retrieve information about queue

tx_queue_performance_info_get 190
Get queue performance information

tx_queue_performance_system_info_get 192
Get queue system performance information

tx_queue_prioritize 194
Prioritize queue suspension list

tx_queue_receive 196
Get message from message queue

tx_queue_send 200
Send message to message queue

tx_queue_send_notify 202
Notify application when message is sent to queue

TWO copy operations are performed for every message transfer

```
TX_QUEUE queue_0;
static uint8_t queue_memory_queue_0[20];

UINT err_queue_0;

// UINT tx_queue_create(TX_QUEUE *queue_ptr, CHAR *name_ptr, UINT message_size, VOID *queue_start, ULONG queue_size);
err_queue_0 = tx_queue_create (&queue_0, (CHAR *) "queue 0", 1, &queue_memory_queue_0, sizeof(queue_memory_queue_0));
```

Example for a queue of size 20 bytes where each message is 1 word long, hence, the capacity is 5 messages.

queue control structure

queue storage structure

SSP code

Sender Task

```
UINT status;
ULONG thread_1_messages_sent;

/* Send message to queue 0.  */
// UINT tx_queue_send(TX_QUEUE *queue_ptr, VOID *source_ptr, ULONG wait_option);
status =  tx_queue_send(&queue_0, &thread_1_messages_sent, TX_WAIT_FOREVER);

/* Check completion status.  */
if (status != TX_SUCCESS)
    break;
```

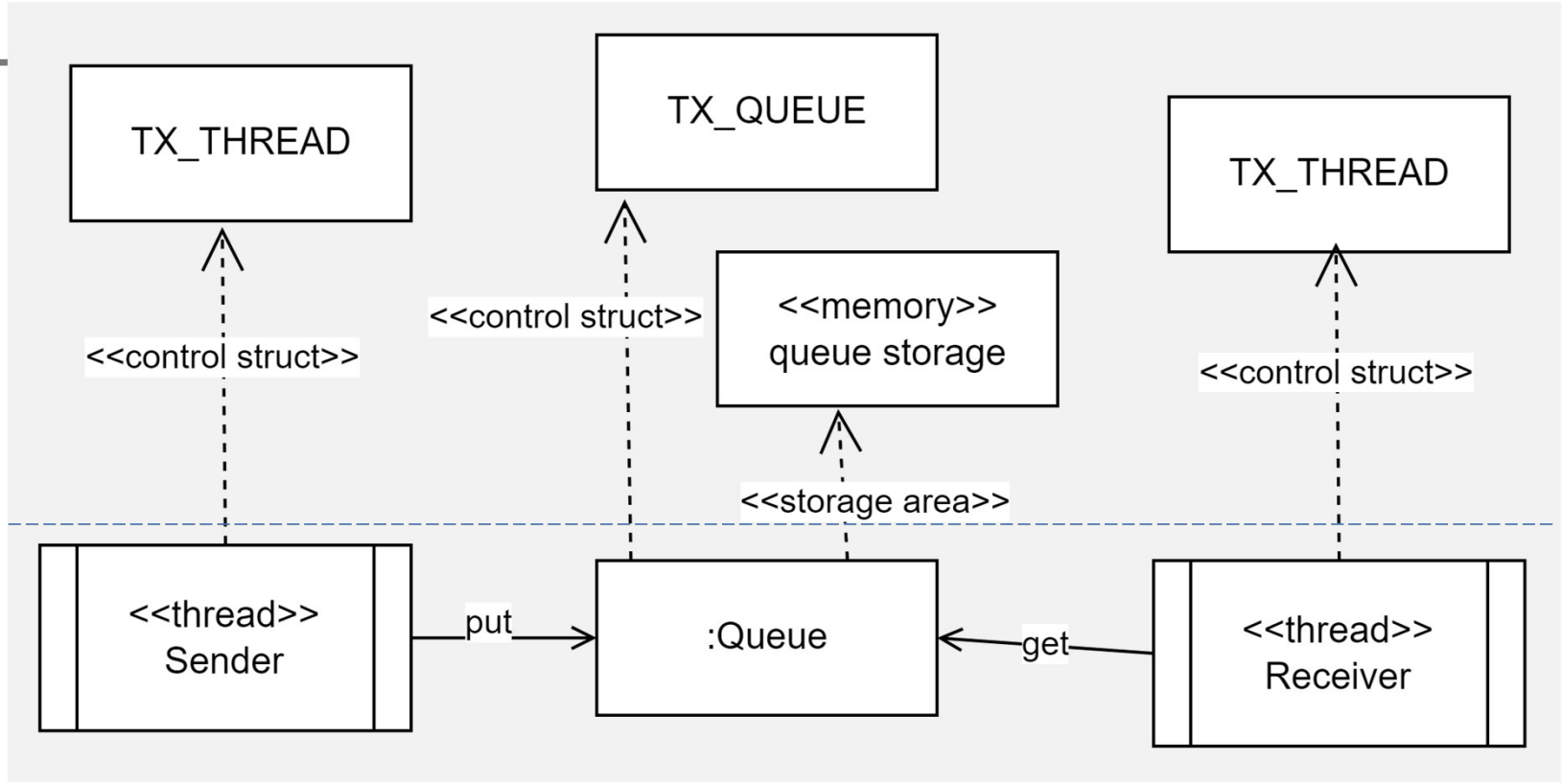
Receiver Task

```
UINT status;
ULONG received_message;
while (1)
{

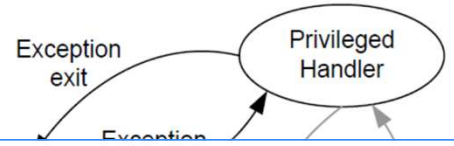
    /* Retrieve a message from the queue.  */
    // UINT tx_queue_receive(TX_QUEUE *queue_ptr, VOID *destination_ptr, ULONG wait_option);
    status = tx_queue_receive(&queue_0, &received_message, TX_WAIT_FOREVER);

    /* Check completion status and make sure the message is what we expected.  */
    if (status != TX_SUCCESS)
        break;
```

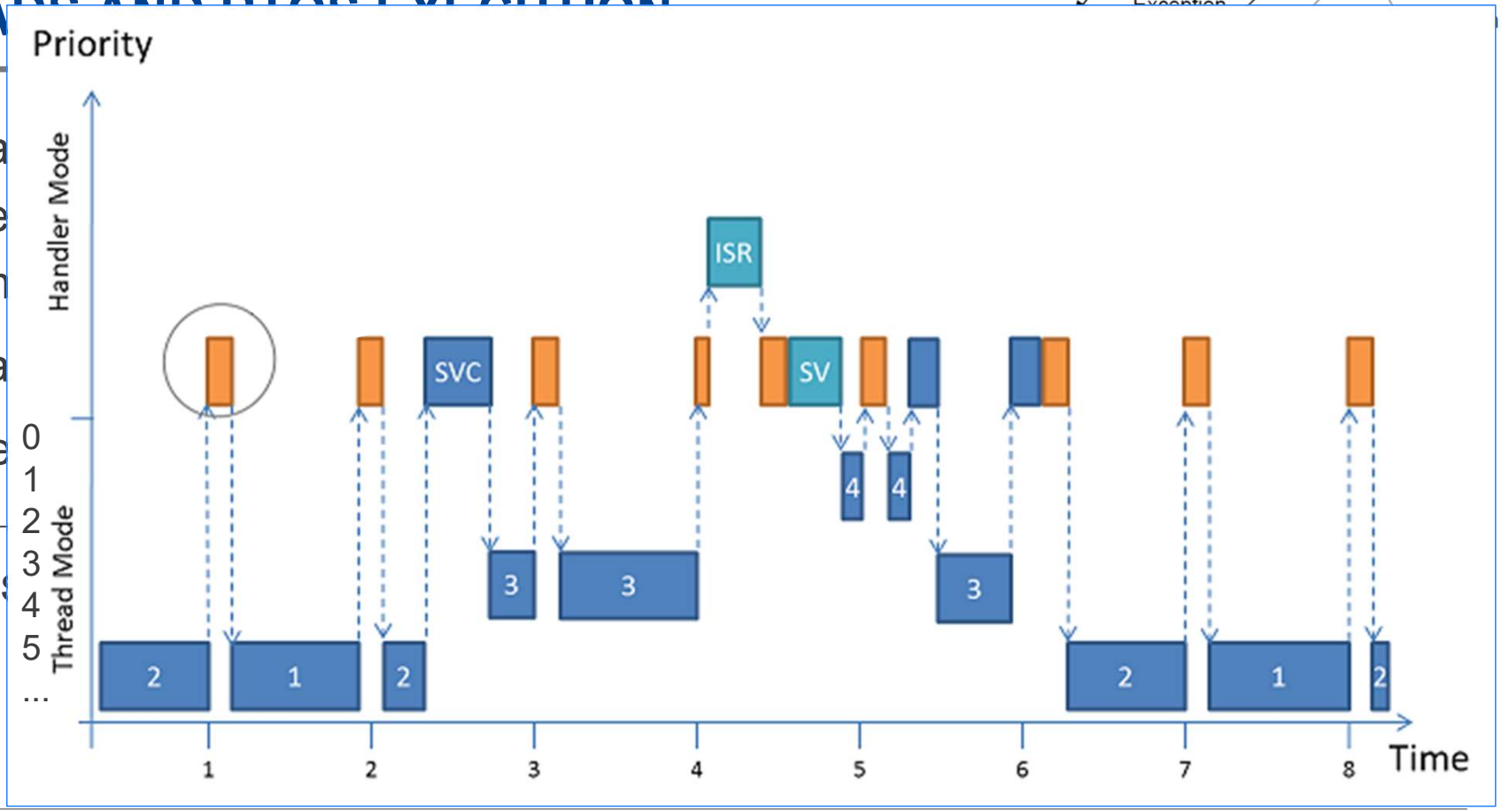
QUEUE INTERNALS



THREADS AND RTOS EXECUTION



- Threa
  Highe
  Preen
- Threa
- Two e
  - ~~**SVC**~~:
  - **Pend**



THREADX INITIALIZATION

INITIALIZATION

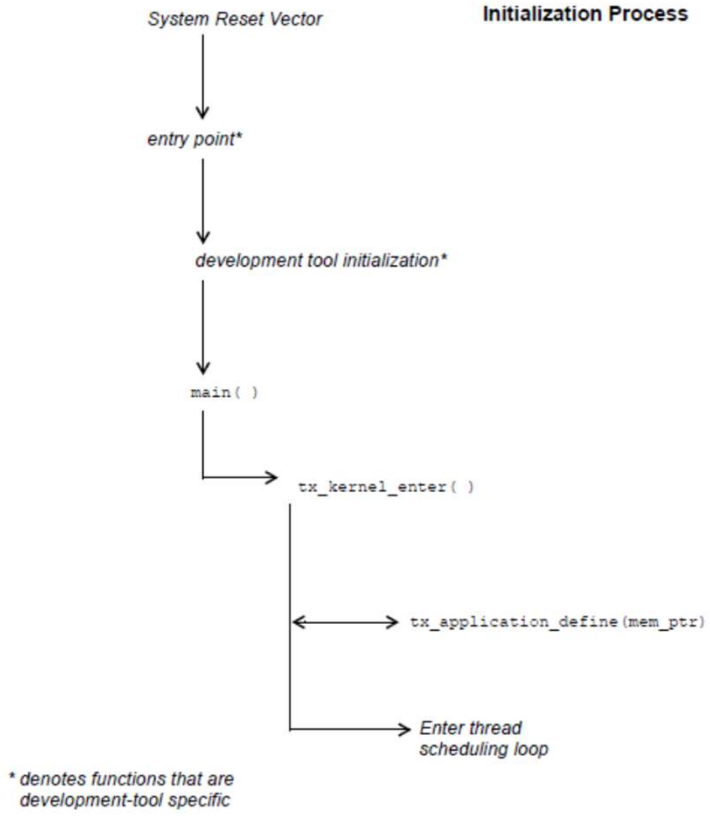


FIGURE 4. Initialization Process

```
#include           "tx_api.h"

unsigned long      my_thread_counter = 0;
TX_THREAD          my_thread;

main( )
{
      /* Enter the ThreadX kernel.  */
      tx_kernel_enter( );
}

void  tx_application_define(void *first_unused_memory)
{

      /* Create my_thread!  */
      tx_thread_create(&my_thread, "My Thread",
          my_thread_entry, 0x1234, first_unused_memory, 1024,
             3, 3, TX_NO_TIME_SLICE, TX_AUTO_START);
}

void   my_thread_entry(ULONG  thread_input)
{
      /* Enter into a forever loop.  */
      while(1)
      {

            /* Increment thread counter.  */
            my_thread_counter++;

            /* Sleep for 1 tick.  */
            tx_thread_sleep(1);
      }
}
```

FIGURE 1. Template for Application Development

INITIALIZATION

```
#include           "tx_api.h"

unsigned long      my_thread_counter = 0;
TX_THREAD          my_thread;

main( )
{
     /* Enter the ThreadX kernel.  */
     tx_kernel_enter( );
}

void  tx_application_define(void *first_unused_memory)
{

     /* Create my_thread!  */
     tx_thread_create(&my_thread, "My Thread",
        my_thread_entry, 0x1234, first_unused_memory, 1024,
          3, 3, TX_NO_TIME_SLICE, TX_AUTO_START);
}

void   my_thread_entry(ULONG  thread_input)
{
     /* Enter into a forever loop.  */
     while(1)
     {

          /* Increment thread counter.  */
          my_thread_counter++;

          /* Sleep for 1 tick.  */
          tx_thread_sleep(1);
     }
}
```

Exception vector table (cstartup_M.s)

__iar_program_start VFP, .data, .bss

App inits, clock, peripherals

create threads
create global ThreadX components

Thread entry functions

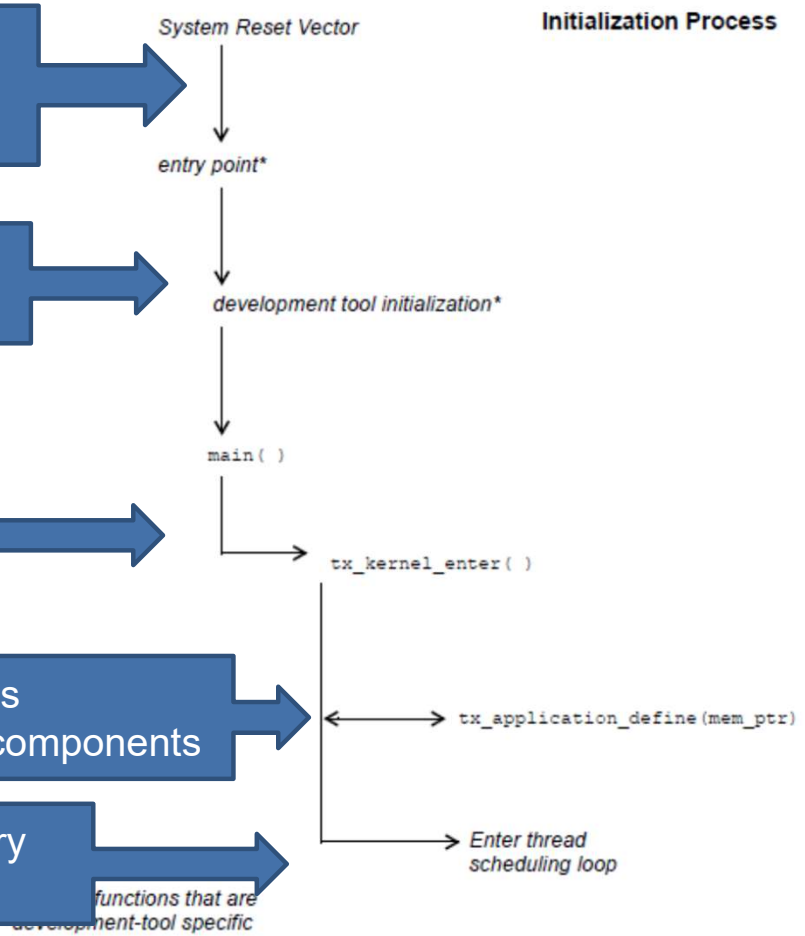


FIGURE 4. Initialization Process

done in SSP

INITIALIZATION

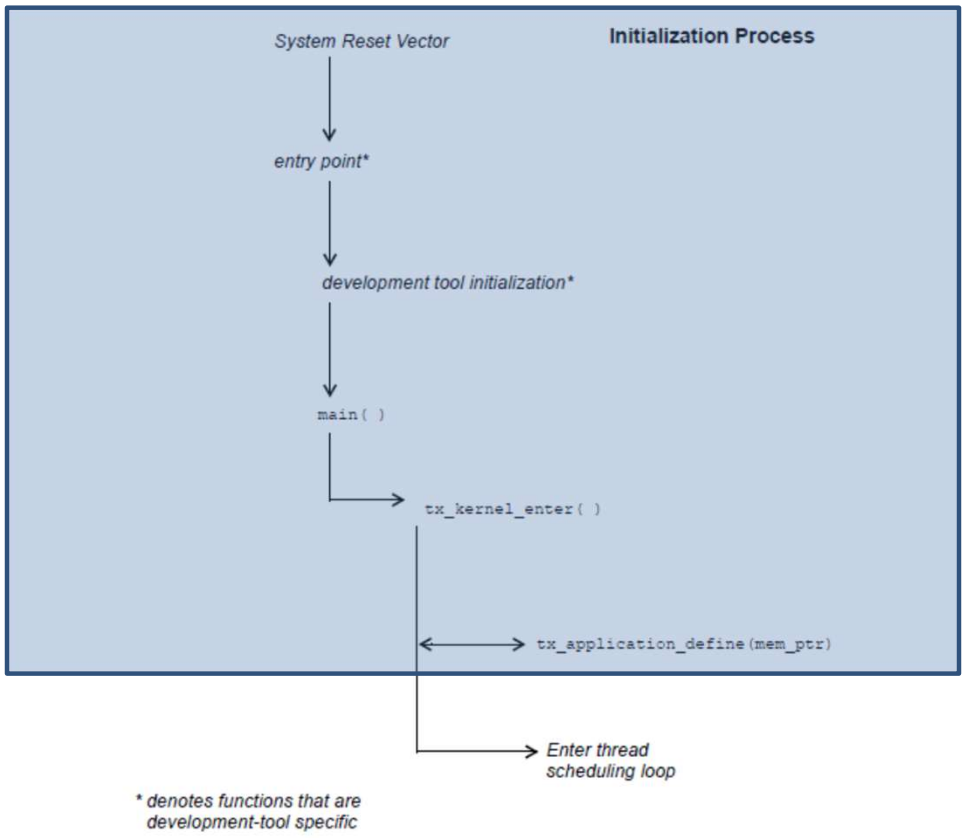


FIGURE 4. Initialization Process

```
#include           "tx_api.h"

unsigned long      my_thread_counter = 0;
TX_THREAD          my_thread;

main( )
{
     /* Enter the ThreadX kernel.  */
     tx_kernel_enter( );
}

void  tx_application_define(void *first_unused_memory)
{

     /* Create my_thread!  */
     tx_thread_create(&my_thread, "My Thread",
         my_thread_entry, 0x1234, first_unused_memory, 1024,
            3, 3, TX_NO_TIME_SLICE, TX_AUTO_START);
}

void   my_thread_entry(ULONG  thread_input)
{
     /* Enter into a forever loop.  */
     while(1)
     {

          /* Increment thread counter.  */
          my_thread_counter++;

          /* Sleep for 1 tick.  */
          tx_thread_sleep(1);
     }
}
```

partially done in SSP

FIGURE 1. Template for Application Development